Mines Paris – PSL
M2 Data Sciences & AI for Finance

How can we reduce the average error rate in RTE's national power demand forecasts by 41%, maintain the same coverage level (95%), while narrowing the prediction intervals by 26%?

Abstract

We examine whether the French system operator's day-ahead electricity-demand forecast can be improved without compromising the information available at forecast time. A leakage-aware, chronological benchmark compares classical time-series models with LightGBM and a multi-resolution Transformer that learn to correct RTE's forecast errors. On the held-out 2026 evaluation, the optimized LightGBM correction reduces MAPE by about 41% while delivering narrower, empirically calibrated 95% prediction intervals. Beyond average accuracy, we assess residual dependence, difficult hours and operational reliability.

Alexandro Bizeul

October 2026

Contents

1 Introduction **1**

2 Data, quality and time conventions **1**
2.1 The dataset . . . 1
2.2 Temporal integrity . . . 2
2.3 Two clocks for the same phenomenon . . . 2
2.4 Chronological split . . . 3

3 Forecasting protocol and metrics **4**
3.1 The operational task . . . 4
3.2 Metrics . . . 4

4 Exploratory analysis **5**
4.1 Level, trend and annual seasonality . . . 5
4.2 The weekly rhythm . . . 6
4.3 School holidays and business slowdowns . . . 8
4.4 Temperature: a real driver, too crude as a raw series . . . 9

5 Naive references and the RTE benchmark **10**

6 Statistical models **10**
6.1 Seasonal decomposition . . . 11
6.2 Stationarity . . . 13
6.3 Identification: ACF and PACF . . . 14
6.4 Estimation and information criterion . . . 15
6.5 Residual diagnostics . . . 15
6.6 D+1 forecasts and Gaussian intervals . . . 16
6.7 SARIMAX: adding heating and cooling degrees . . . 17
6.8 Holt–Winters . . . 17
6.9 Statistical models: summary . . . 18

7 From statistical tables to LightGBM **18**
7.1 Gradient boosting as residual learning . . . 18
7.2 Calendar feature engineering . . . 19
7.3 Load and weather lags . . . 20
7.4 RTE features and standardisation . . . 20
7.5 Shared training protocol . . . 20
7.6 Direct forecasting . . . 21
7.7 Local trend and residual . . . 21
7.8 Correcting the RTE forecast error . . . 21
7.9 Hyperparameter search . . . 22
7.10 Global comparison on validation . . . 23

8 Final evaluation on 2026 **23**
8.1 Hourly conformal prediction intervals . . . 24
8.2 Residual diagnostics . . . 24
8.3 When the models fail . . . 26
8.4 Consolidated table . . . 28

9 Outlook: a multi-resolution Transformer **28**
9.1 Why a Transformer, and why multi-resolution? . . . 29
9.2 Problem formulation . . . 29
9.3 Branch-specific tokenization . . . 30
9.4 Attention and relative positional encoding . . . 30
9.5 Fusing branch representations . . . 30
9.6 Adaptation to French D+1 forecasting . . . 31
9.7 First result on 2025 . . . 32
9.8 Tuning and retraining . . . 33
9.9 2026 test set . . . 33

10 Conclusion **35**

1 Introduction

In France, RTE publishes every day a forecast of national electricity demand for the following day. This D+1 forecast guides generation scheduling, cross-border exchanges and operating margins. An error of a few percent, applied to a demand of about 50 000 to 80 000 MW, amounts to several gigawatts.

The goal of this project is not to replace RTE's internal models, whose covariates and architecture are unknown. The goal is to start from the published forecast, measure its historical errors, and learn to correct them. Rather than reconstructing demand from scratch, we look for remaining signal in the residual

$$e_t^{\mathrm{RTE}} = y_t - \hat{y}_t^{\mathrm{RTE}}, \tag{1}$$

where y_t is observed national consumption at time t (in MW) and $\hat{y}_t^{\mathrm{RTE}}$ is the corresponding D+1 forecast. A residual is not a "score" assigned to the model: it is the signed gap between reality and the forecast. If that gap contains repeated structure (hour-of-day bias, recent error, calendar effects), a second model can estimate it and subtract it.

This strategy is valid only under three conditions. First, the RTE forecast must be available at the chosen origin: 11:00 Paris time on day D. This assumption is not checked against the exact publication schedule; it is therefore a methodological limitation. Second, explanatory variables (load lags, temperature, calendar) must be built without information leakage, i.e. from data strictly earlier than 11:00. Leakage means using, to forecast D+1, a quantity that the operator would not have had at 11:00 on day D. Third, model comparison must be fair: same origin, same target hours, same daylight-saving conventions, same evaluated observations.

The document follows the work in order. We start with the data, their quality and the dual timezone (UTC for technical integrity, Europe/Paris for the calendar and the origin). We define the forecasting protocol, then explore the series. We build naive references, apply the Box–Jenkins methodology (decomposition, stationarity, identification, estimation, diagnostics, forecasting), enrich SARIMA with temperature, and compare with Holt–Winters. We then move to LightGBM: direct forecasting, trend–residual decomposition, then RTE error correction. We tune, we compare, and we open the 2026 test only after the model is selected. We finish with residual diagnostics, failure analysis, and the training and evaluation of a multi-resolution Transformer.

> **Scope of the paper**
>
> This text is self-contained: tests, architectures and modelling choices are described with sentences, formulae, tables and diagrams, without code. Figures come from the study. The deep-learning part describes the MTST architecture and its subsequent empirical evaluation.

2 Data, quality and time conventions

2.1 The dataset

The series under study is hourly, national, and covers the period from 1 January 2019 at 00:00 UTC to 30 June 2026 at 21:00 UTC. It contains 65 710 rows, which is exactly the number of UTC hours expected over this interval. Each observation gathers four quantities: the UTC timestamp, realised

consumption y_t (in MW), the RTE D+1 forecast $\hat{y}_t^{\mathrm{RTE}}$ (in MW), and a temperature T_t (in °C).

Table 1: *Descriptive statistics after interpolation of the seven missing values (full series).*

Quantity	Mean	Std. dev.	Range
Consumption (MW)	51 497	11 019	29 364 – 90 210
RTE D+1 forecast (MW)	50 830	11 048	27 712 – 89 975
Temperature (°C)	13.07	7.02	−5,09 – 38,52

Two immediate readings follow. On the one hand, national consumption oscillates over a very wide range, from about 29 GW at the summer trough to more than 90 GW at the winter peak: the phenomenon to be forecast is not a small fluctuation around a level, it is a large-amplitude seasonal cycle. On the other hand, the RTE forecast already has, on average, the right order of magnitude (slightly below the realisation), which makes the goal of improving it all the more demanding.

2.2 Temporal integrity

Before any modelling, we check that the series clock is sound. There are no timestamp duplicates, no gaps in the UTC index, and the number of rows coincides with the number of elapsed hours. However, seven consumption and RTE forecast observations are missing. They all fall, without exception, on the October transition to winter time, around 00:00 UTC, that is at the moment when the French civil clock repeats 02:00. Temperature itself has no gaps.

These seven absences are therefore not random outages: they are the symptom of a daylight-saving convention poorly aligned across sources. To preserve a continuous hourly series, which is indispensable for lags and autoregressive models, these seven points are linearly interpolated in time. Originally missing observations are nevertheless *excluded from every final evaluation*: a model is never evaluated against an interpolated truth.

2.3 Two clocks for the same phenomenon

Electricity consumption follows human activity, and human activity follows civil time, not UTC. At 19:00 in Paris, households cook, light their homes and return from work — whether in January or in July. Yet 19:00 in Paris is 18:00 UTC in winter (UTC+1) and 17:00 UTC in summer (UTC+2). If hourly profiles are averaged in UTC, one therefore compares different moments of daily life according to the season. Figure 1 shows this unambiguously: in UTC, the winter evening peak and the summer evening peak do not fall at the same abscissa; in Paris time, they overlay.

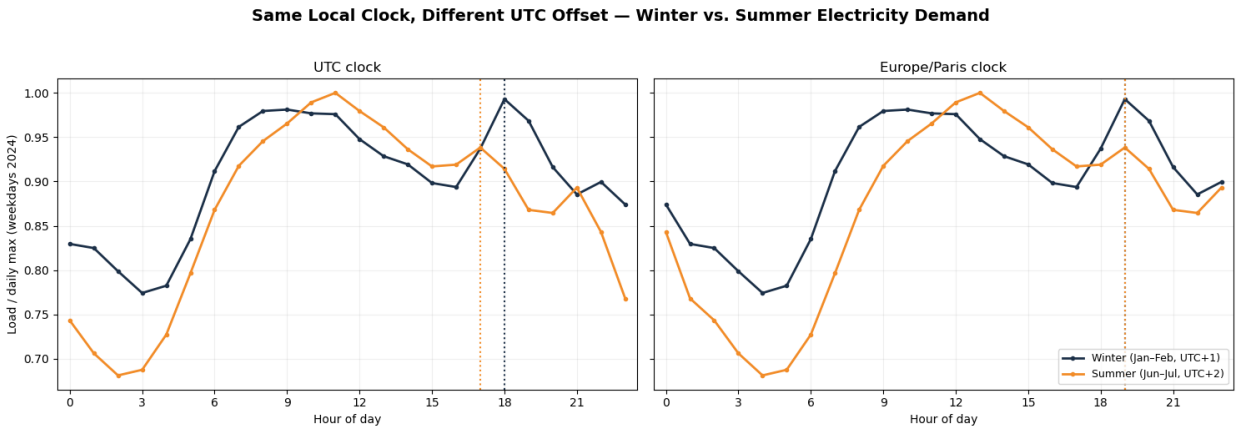


Figure 1: *Normalised hourly demand profiles, working days 2024. On the left, the UTC clock artificially shifts winter and summer. On the right, the Europe/Paris clock aligns civil hours and makes evening peaks comparable.*

The methodological decision is therefore twofold, and it structures the rest of the project.

Dual-timezone strategy

All observations remain indexed in **UTC**. This is the clock that guarantees unique timestamps, a regular hourly step, and lags of 48 h or 168 h that truly measure 48 or 168 elapsed hours, including across a clock change. By contrast, calendar variables, the forecast origin (11:00), public holidays and the 24 target slots of the following day are defined in **Europe/Paris**. The two representations do not compete: one protects technical integrity, the other protects operational meaning.

Daylight-saving transitions are treated explicitly, because they break the illusion of a 24-hour civil day. At the March transition, one local hour disappears (typically 02:00): the model nevertheless produces 24 forecasts indexed from 0 to 23, and the one with no civil existence is discarded. At the October transition, 02:00 occurs twice: a missing value is assigned to the first occurrence and the forecast for the second is kept. These rules, known in advance, introduce no information leakage. They apply to all models, without exception.

2.4 Chronological split

A time series cannot be split by random sampling. Mixing past and future, including through cross-validation, would amount to training with information that the operator would not have at 11:00. The split is therefore strictly chronological. The 2026 test is opened only after the model is chosen definitively.

Table 2: *Temporal split of the project. Sizes correspond to hourly UTC observations.*

Set	Period	Size	Role
Training	2019–2024	52 607	Training and feature engineering
Hyperparameter tuning	2019–2023 / 2024	—	Chronological optimisation
Validation	2025	8 760	Comparison and selection
Conformal calibration	Jan.–Jun. 2025	—	Hourly interval margins
Conformal evaluation	Jul.–Dec. 2025	—	Calibration check
Held-out test	Jan.–Jun. 2026	4 343	Out-of-sample performance



Figure 2: *Protocol timeline. The second half of 2025 is used to evaluate conformal intervals; 2026 is never used to choose a model.*

3 Forecasting protocol and metrics

3.1 The operational task

All models in this project answer the same question, asked at the same time. On day D, at 11:00 Paris time, one must produce 24 numbers: the national consumption forecast for each civil hour of the following day, from 00:00 to 23:00. The forecast origin, denoted $t_0(D)$, is therefore 11:00 Europe/Paris on day D, converted to UTC (10:00 UTC in winter, 09:00 UTC in summer). Historical variables may use only observations strictly earlier than $t_0(D)$.

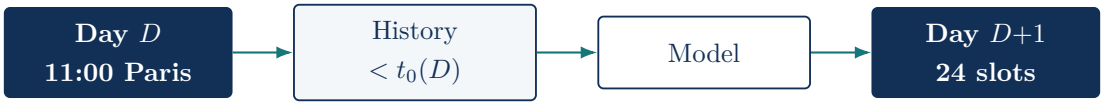


Figure 3: *Protocol common to all models. The origin is civil (11:00 Paris); lags are technical (UTC); the output is a vector of 24 hourly forecasts.*

This protocol, sometimes called daily-step *walk-forward*, is more constraining than an hour-by-hour rolling forecast. In particular, it forbids using consumption on D after 11:00 to forecast D+1, and it imposes an effective horizon ranging from about 13 hours (00:00 slot of D+1) to 37 hours (23:00 slot). This is precisely the constraint an operator faces.

3.2 Metrics

The main selection metric is MAPE (Mean Absolute Percentage Error), that is the mean relative error in percent. Intuitively, a MAPE of 3% means that, on average, the forecast departs from realised consumption by 3%. French national consumption is strictly positive and far from zero, which avoids the classical unstable-denominator problem. MAPE is complemented by two metrics in MW. MAE (Mean Absolute Error) is the mean absolute gap, directly readable as power. RMSE

(Root Mean Squared Error) penalises large errors more heavily, because gaps are squared before averaging.

$$\text{MAPE} = \frac{100}{n}\sum_{t\in\mathcal{E}}\left|\frac{y_t-\hat{y}_t}{y_t}\right|, \qquad \text{MAE} = \frac{1}{n}\sum_{t\in\mathcal{E}}|y_t-\hat{y}_t|, \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum_{t\in\mathcal{E}}(y_t-\hat{y}_t)^2}. \tag{2}$$

The evaluation set $\mathcal{E}$ is identical for all models. It excludes originally missing observations, the first occurrence of 02:00 at the October transition, and any interpolated truth.

When forecast intervals $[\hat{\ell}_t, \hat{u}_t]$ are available, two additional quantities are reported. *Empirical coverage* is the share of observations that actually fell inside the interval,

$$\widehat{C}_{1-\alpha} = \frac{1}{n}\sum_{t\in\mathcal{E}}\mathbf{1}\{y_t \in [\hat{\ell}_t, \hat{u}_t]\}. \tag{3}$$

For a target $\alpha = 0{,}05$, one aims at 95%. The *mean width* $\frac{1}{n}\sum_t(\hat{u}_t - \hat{\ell}_t)$ measures displayed uncertainty, in MW. These two quantities must be read together. A very wide interval easily covers 95% of the points, but it is uninformative. A narrow interval is useful only if it retains the target coverage. The operational objective of this work is precisely that of the title: reduce mean error, keep 95% coverage, and shrink the intervals.

4 Exploratory analysis

This section identifies the temporal structures that every model will have to capture, and those that a calendar or a temperature series can explain. Without this diagnosis, the calendar scores, lags and degree-days introduced later would have no justification.

4.1 Level, trend and annual seasonality

Figure 4 overlays the raw hourly series and moving averages of increasing length, from 24 hours to 365 days. Two facts stand out. First, hourly noise is considerable, but it fades as soon as one aggregates: under the volatility lies a very regular annual seasonality, with a winter peak and a summer trough. Second, the annual mean (purple curve) is remarkably flat over 2019–2024. There is no strong long-run trend of the structural-growth or structural-decline type. Year-to-year variations exist, but the background level remains of the order of 50 GW.

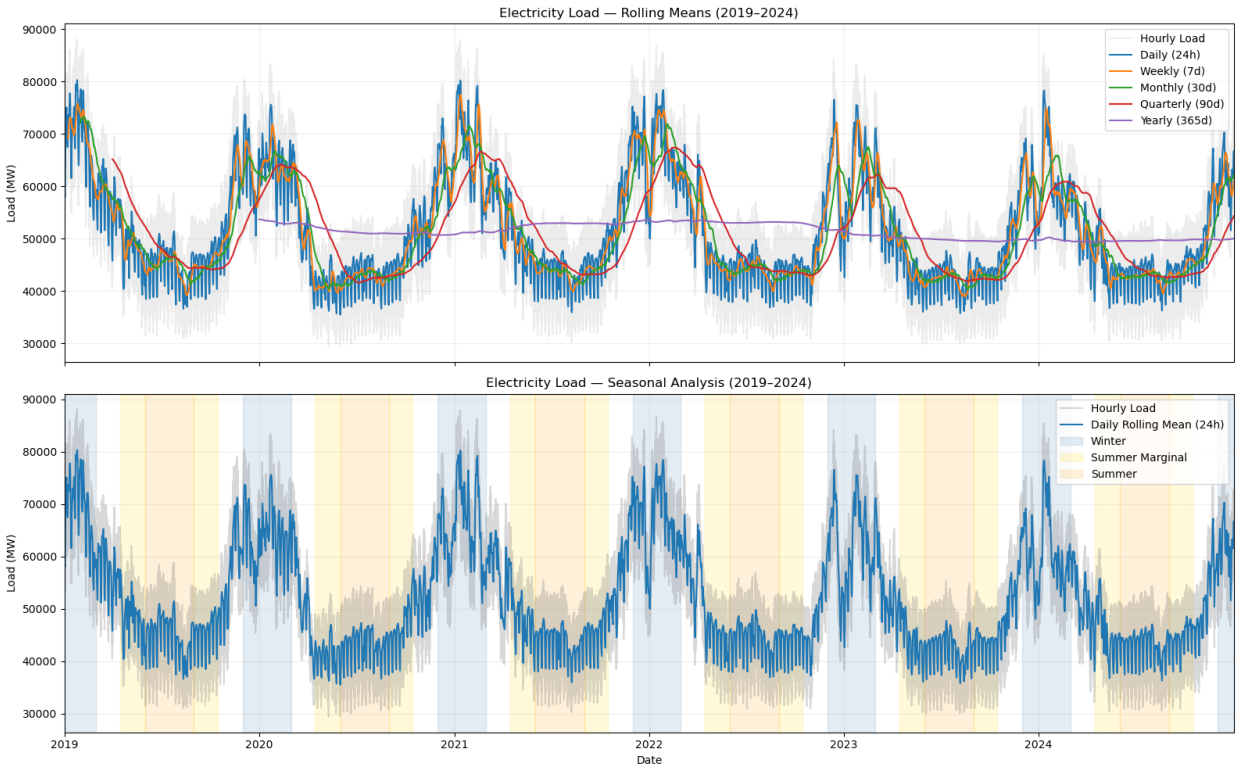


Figure 4: *Top: moving averages of load, 2019–2024. Bottom: highlighting of civil seasons, with a "marginal summer" period (late spring, early autumn) where demand stays low even though the calendar is no longer strictly summer.*

The lower panel refines the seasonal reading. Classifying the year into winter and summer is too coarse for a temperate climate. In April–May and in September–October, temperatures are often mild enough to limit residential heating, and consumption behaves more like summer than like winter. This observation will later motivate a four-level seasonal score rather than a simple winter/summer indicator, as well as heating and cooling degree-days.

4.2 The weekly rhythm

Zooming in on a winter week (mid-January 2024, Figure 5) reveals the second layer of seasonality: the intra-day cycle. At night, the curve troughs. In the morning, it rises sharply. The afternoon marks a relative trough, then the evening produces a second peak, often the most pronounced in winter. These two surges — 05:00–08:00 and 17:00–19:00 — correspond to residential tariff peak hours, and they will enter the hourly score as such.

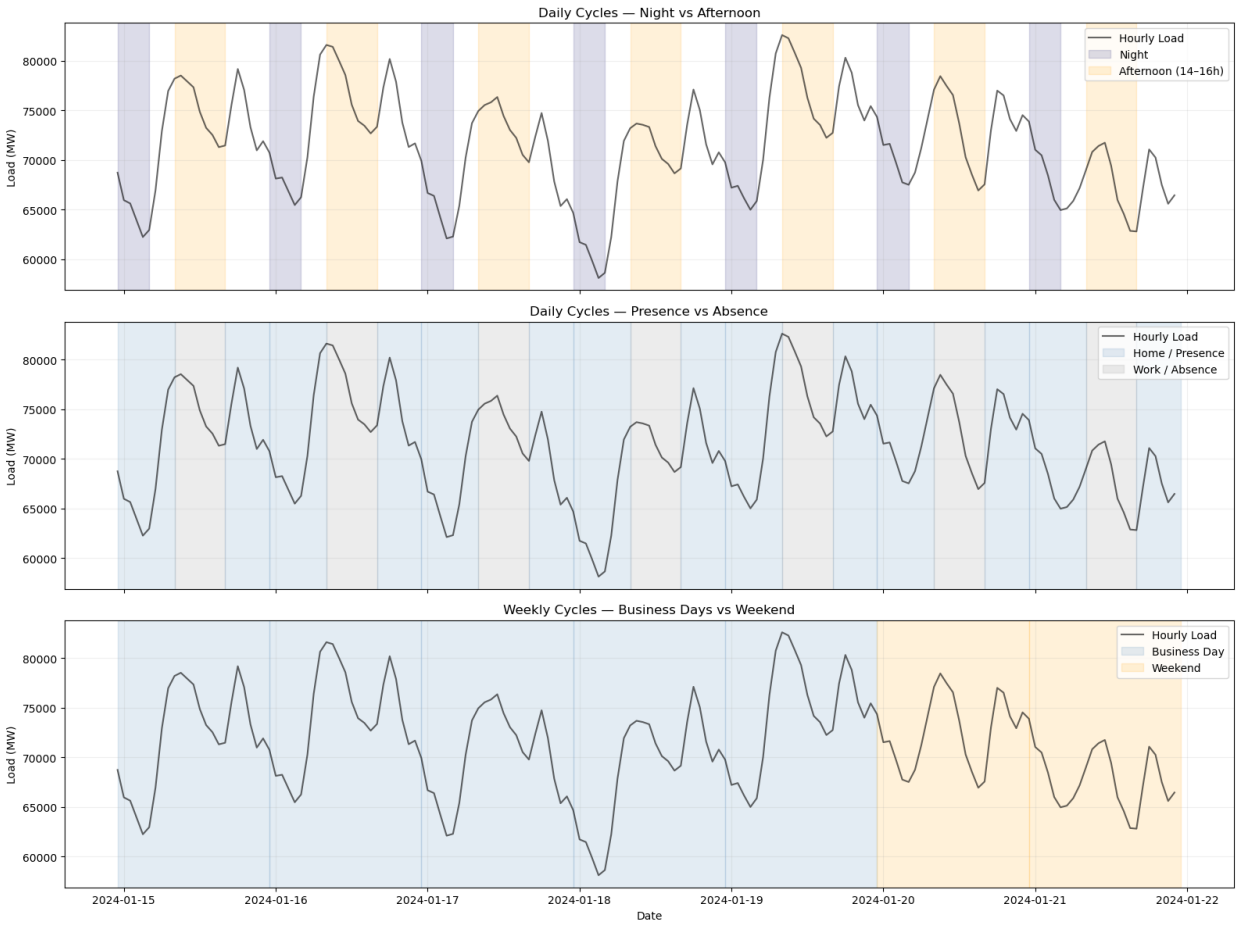


Figure 5: *A winter week. Top: night-time troughs and morning and evening peaks. Middle: presence at home (evening–night–morning) does not by itself distinguish peaks from troughs. Bottom: at the weekend, national demand* falls*, contrary to the intuition of higher residential occupancy.*

The middle panel tests a naive hypothesis: if households are away during office hours, consumption should fall, and if they are present in the evening, it should rise. This reading is only partly true. In the evening and at night, presence and inactivity overlay: people are at home, but they sleep. An *activity* indicator (tariff peak, working day, business holidays) will be more discriminant than a simple presence indicator.

The lower panel holds the most useful surprise of the diagnosis. At the weekend, while dwellings are more occupied, *national* demand recedes. This is not an artefact of one particular week. Figure 6 confirms it on a winter quarter and a summer quarter: the Saturday–Sunday drop is generalised, whatever the season.

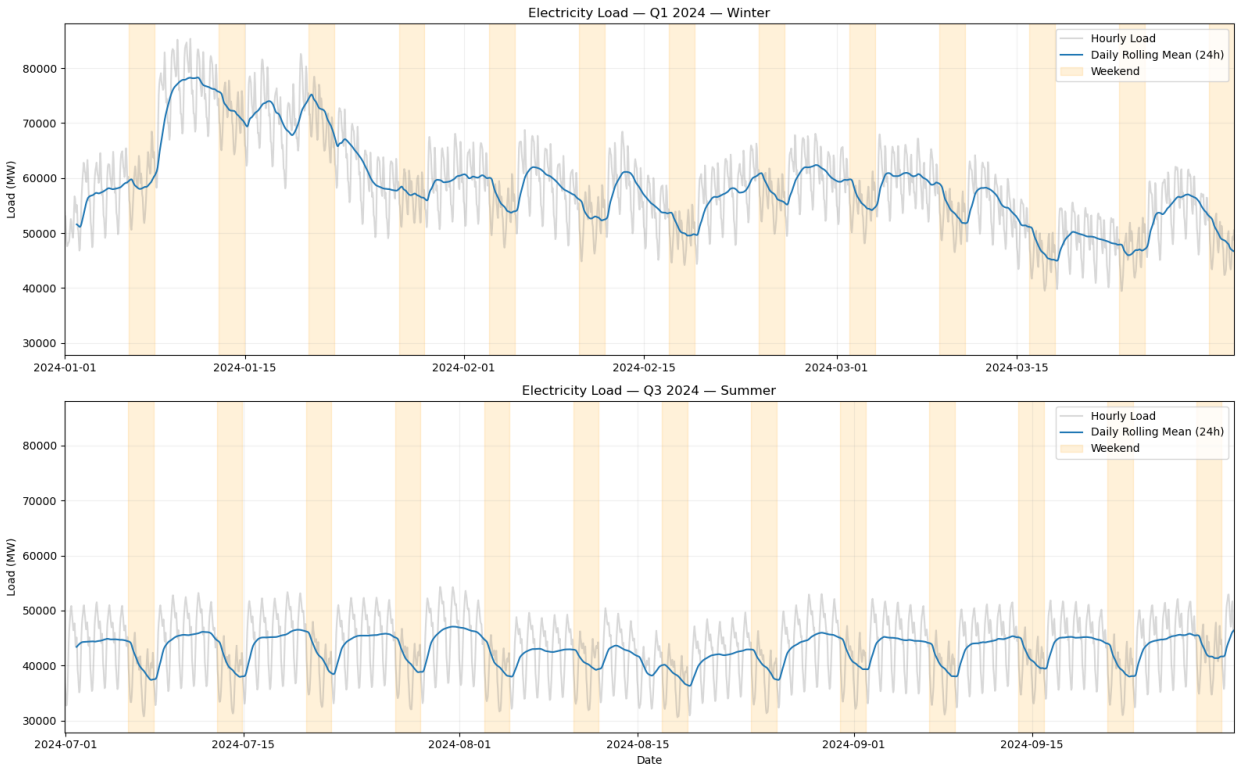


Figure 6: *First quarter 2024 (winter) and third quarter (summer). Orange bands mark weekends. The drop is systematic there, including in summer where heating explains nothing.*

Two drivers of different natures are therefore identified. **Economic and industrial activity** governs the aggregate weekly level: factories, offices, transport and data centres are used less on Sunday. **Household activity** governs the intra-day shape: the winter evening peak exists only because millions of dwellings heat, cook and light at the same time. A good model must carry both, and not confuse them.

4.3 School holidays and business slowdowns

If the weekend lowers the curve, holidays should lower it further, but not all in the same way. Figure 7 contrasts school holidays (zones A, B, C) and periods of business slowdown (around Christmas and August). School holidays, staggered and shifted by zone, blur the signal. Business holidays, more concentrated, match the large national troughs much better, in particular around the year-end holidays. In summer, a low plateau sets in during July–August, consistent with a partial shutdown of productive capacity, without the same "clean stop" observed in December.

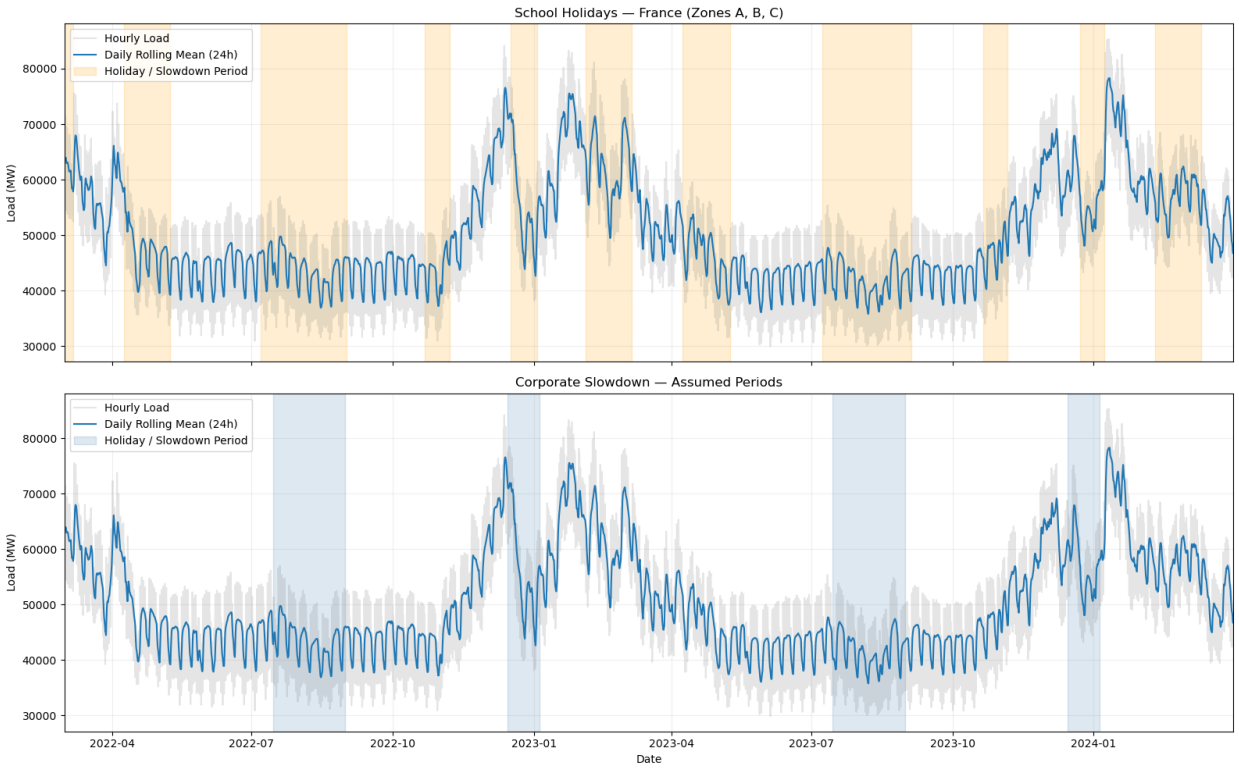


Figure 7: *School holidays (top) and assumed business-slowdown periods (bottom). The business calendar explains the year-end national drop better than the school calendar alone.*

This distinction will feed the calendar score: a weekend day, a school-holiday day and a business-holiday day will not carry the same weight.

4.4 Temperature: a real driver, too crude as a raw series

Figure 8 overlays load and extreme days, defined by empirical quantiles of daily temperature: hot days when the maximum exceeds 23,93 °C (quantile 0.80), cold days when the minimum falls below 4,05 °C (quantile 0.20). The link is visible — winter peaks coincide with blue, summer troughs with red — but it is too crude to enter a model as a raw series. Hourly temperature from a station (or an aggregate) does not "speak" to demand in the same way at 3 °C and at 18 °C, nor at 18 °C and at 32 °C. Below one threshold, one heats; above another, one cools; in between, sensitivity weakens.

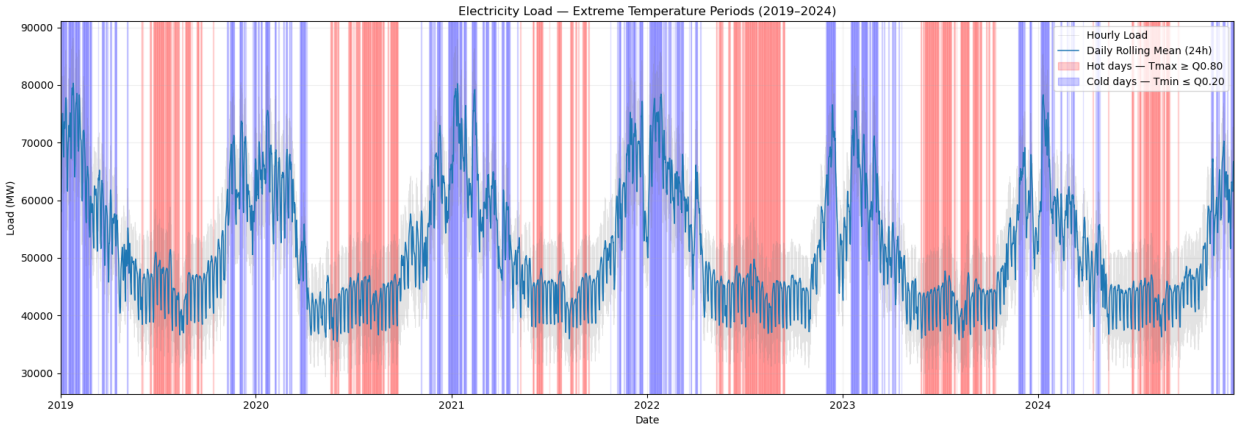


Figure 8: *Periods of extreme temperature, 2019–2024. Cold (blue) accompanies the highest load levels; heat (red) the lowest, with a few summer cooling peaks.*

Hence the idea, classical in energy analysis, of degree-days. They will be defined later, not as daily cumulatives, but as hourly gaps to two thresholds, 15 °C for heating and 22 °C for cooling, computed on temperature lagged by 48 hours in order to remain compatible with the forecast origin.

5 Naive references and the RTE benchmark

Before estimating any parameter, we establish baselines. A model that does not beat a copy of the previous week has no operational legitimacy. Three references are built under the protocol of the previous section.

The **48-hour naive** forecast copies consumption observed two days earlier, at the same UTC instant. The 48-hour shift is not arbitrary: at 11:00 on day D, the last complete civil day available is not D, it is $D-1$, and the homologous slot of $D+1$ lies 48 hours before the start of $D+1$. The **168-hour seasonal naive** forecast copies the same hour of the previous week, which captures the weekly cycle in one step. Finally, the **RTE D+1** forecast is the operational benchmark, the one we seek to beat.

More distant naive forecasts (720 hours, i.e. one month; 2160 hours, i.e. one quarter) are computed for the record. They collapse, which will surprise no one: copying January to forecast February, or autumn to forecast winter, makes no sense on a series with strong annual seasonality.

Table 3: *Reference performances on the 2025 validation set (8 759 hours evaluated).*

Model	MAPE (%)	MAE (MW)	RMSE (MW)
RTE D+1	**2,82**	**1397**	**1726**
Seasonal naive 168 h	6,43	3461	5146
Naive 48 h	8,04	3994	5219
Monthly naive 720 h	12,70	6437	8306
Quarterly naive 2160 h	22,20	11234	14317

The weekly seasonal naive is retained as the statistical baseline: it is the simplest model that still "understands" the weekly rhythm. But the gap with RTE is already a warning. A MAPE of 2.82% is a professional forecast, fed by information we do not have (weather forecasts, industrial schedules, incidents, etc.). Any model that ignores this forecast starts with a handicap. Any model that takes it as an input starts, on the contrary, from an excellent base.

6 Statistical models

This section applies, step by step, the Box–Jenkins methodology. We decompose, stationarise, identify, estimate, diagnose and forecast. The model is then enriched with temperature regressors, then compared with Holt–Winters exponential smoothing. The aim is not only to produce a number: it is to understand *why* a SARIMA, even well specified, remains far from RTE.

6.1 Seasonal decomposition

The series is first written as a sum of components. STL decomposition (Seasonal-Trend decomposition using Loess) isolates a slow trend, a seasonality and a residual. Applied to hourly load 2022–2024 with a daily seasonal period, it confirms what the eye had seen: a trend that follows winter and summer, an intra-day seasonality of amplitude of the order of ±10 GW, and a residual that is not white noise — structure remains, in particular around holidays and cold spells (Figure 9).

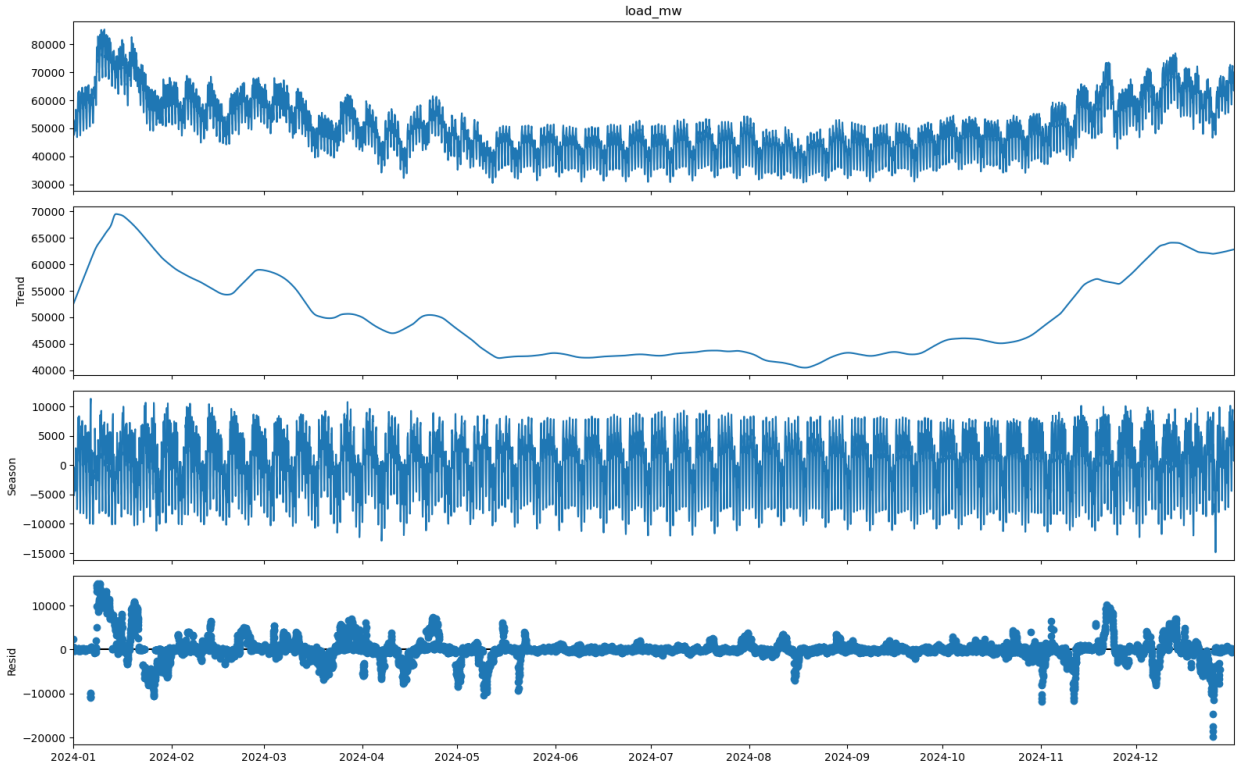


Figure 9: *STL decomposition of hourly load over 2024 (daily seasonal period). The trend carries the annual cycle; seasonality carries the intra-day cycle; the residual remains structured in winter.*

Because the series has at least two seasons (the day and the week), we move to MSTL, which stacks a seasonality of period 24 and a seasonality of period 168 (Figure 10). Comparing these two components on four representative months of 2024 — January, April, July, October — shows that daily amplitude is not constant through the year (Table 4). The intra-day range is maximal in January (20 034 MW) and in July (17 145 MW), and weaker in spring. Weekly amplitude is of the same order in winter and spring, and a little weaker in summer. A SARIMA with a single seasonality and constant coefficients will therefore struggle to tell the whole story.

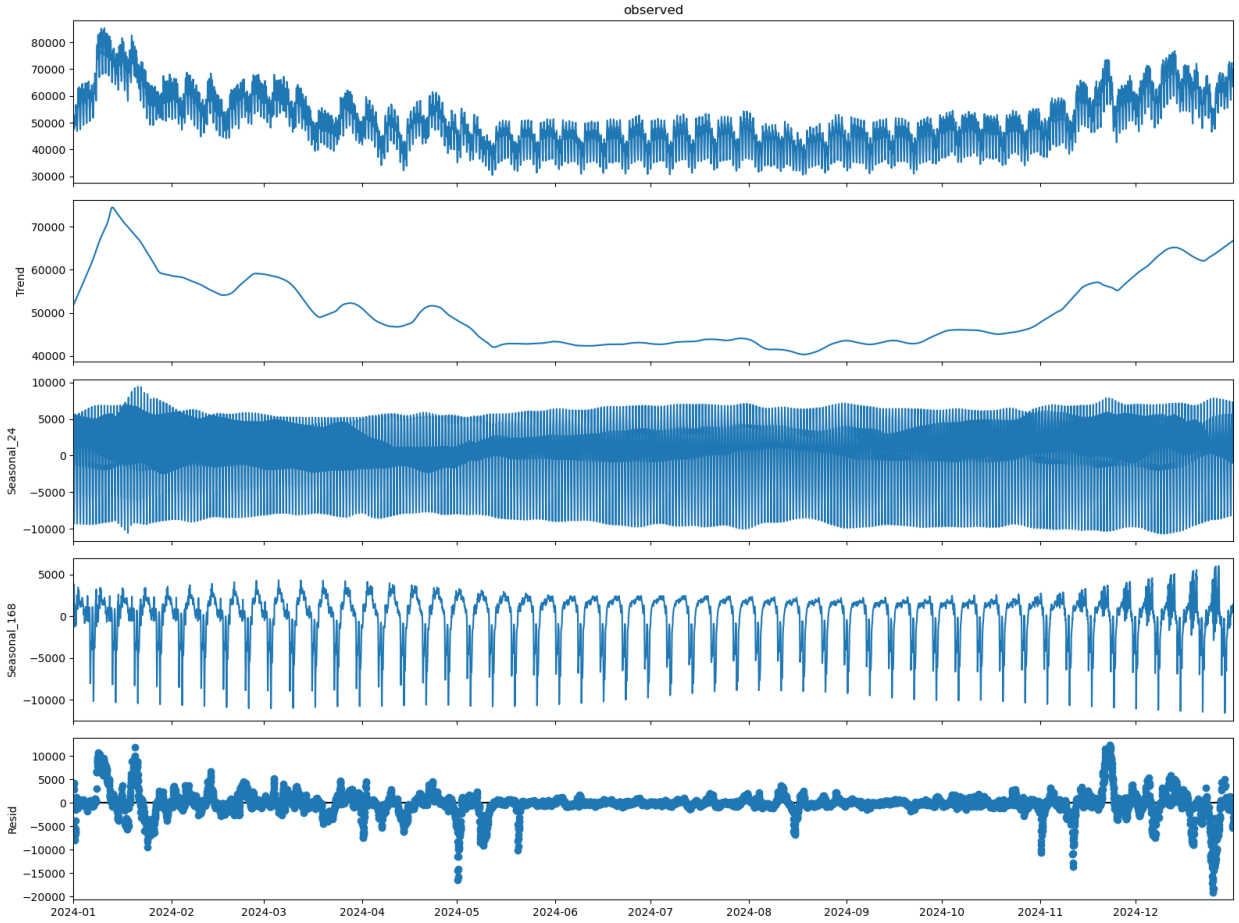


Figure 10: *MSTL decomposition of hourly load. Besides the annual trend, two seasons coexist: the intra-day cycle (24 h) and the weekly cycle (168 h). The residual is quieter than under simple STL, but it is still not white.*

Table 4: *Amplitude of MSTL seasonal components, four months of 2024.*

Season	Daily std. dev. (MW)	Weekly std. dev. (MW)	Daily range (MW)	Weekly range (MW)
Winter — January	4794	2511	20 034	14 383
Spring — April	4056	3086	14 454	14 888
Summer — July	5170	2860	17 145	11 929
Autumn — October	4947	2602	16 519	13 013

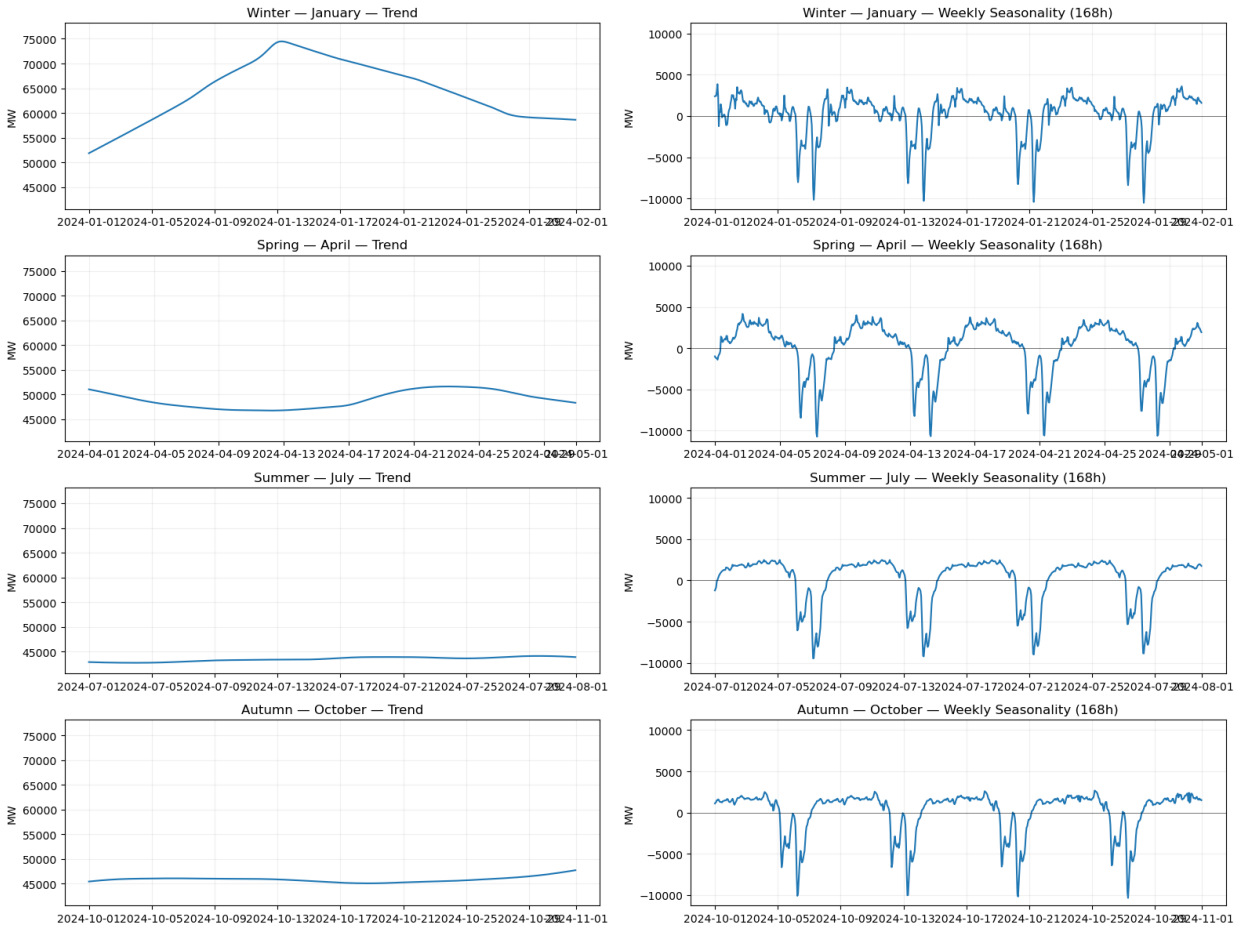


Figure 11: *Daily (left) and weekly (right) seasonal components extracted by MSTL, for four months of 2024. The amplitude of the daily cycle varies clearly from one season to another, which a SARIMA with constant coefficients cannot fully absorb.*

6.2 Stationarity

A process (y_t) is (weakly) stationary if its expectation, variance and autocovariances do not depend on time. ARMA models, and therefore SARIMA after differencing, rest on this assumption. Two complementary tests are applied.

The Augmented Dickey–Fuller (ADF) test takes as null hypothesis the presence of a unit root. Rejecting H_0 (small p-value) argues for the absence of a unit root. The KPSS test takes the inverse hypothesis: the series is stationary around a level. *Not* rejecting H_0 (large p-value) argues for stationarity. The two tests together avoid relying on a single verdict.

On the raw series, the conflict is classical for large-amplitude seasonal series. ADF rejects the unit root (statistic $-4{,}10$, p-value $0{,}001$). KPSS rejects level stationarity (statistic $3{,}51$, p-value $\leq 0{,}01$). The series is therefore not simple noise around a mean, but it is not an ordinary random walk either. We difference.

Let B be the lag operator, $By_t = y_{t-1}$. Ordinary differencing of order d and seasonal differencing of order D and period s are written

$$\nabla^d \nabla_s^D y_t = (1 - B)^d (1 - B^s)^D y_t. \tag{4}$$

Table 5: *Stationarity tests after transformations. "Supported" means: ADF rejects H_0 and KPSS does not reject it.*

Transformation	ADF p	KPSS p	ADF rejects	Stationarity supported
Raw series	0,001	0,010	yes	no
∇ (diff. 1)	$< 10^{-3}$	0,10	yes	**yes**
∇^2	$< 10^{-3}$	0,10	yes	yes
∇_{24}	$< 10^{-3}$	0,10	yes	**yes**
∇_{168}	$< 10^{-3}$	0,045	yes	no
$\nabla\nabla_{24}$	$< 10^{-3}$	0,10	yes	yes
$\nabla\nabla_{168}$	$< 10^{-3}$	0,10	yes	yes

First differencing and daily seasonal differencing each suffice, on their own, to reconcile ADF and KPSS. Weekly differencing alone does not suffice. Differencing twice (ordinary, or ordinary and seasonal) also stationarises, but at the cost of a risk of *over-differencing*, which injects a non-invertible MA component and complicates estimation. Two main candidates are therefore retained, ∇y_t and $\nabla_{24} y_t$, and their ACF and PACF will be read.

6.3 Identification: ACF and PACF

A SARIMA$(p, d, q) \times (P, D, Q)_s$ process is written

$$\phi(B)\,\Phi(B^s)\,\nabla^d \nabla_s^D y_t = \theta(B)\,\Theta(B^s)\,\varepsilon_t, \tag{5}$$

where ϕ and θ are the non-seasonal autoregressive and moving-average polynomials, Φ and Θ their seasonal analogues, and (ε_t) white noise. The ACF of a pure MA vanishes after lag q; the PACF of a pure AR vanishes after lag p. In the presence of seasonality, the same signatures repeat at multiples of $s = 24$.

Reading the diagrams (Figures 12 and 13) is never unambiguous on a series this long and this seasonal. Candidate grids are drawn from them, kept compact on purpose, rather than a single model.

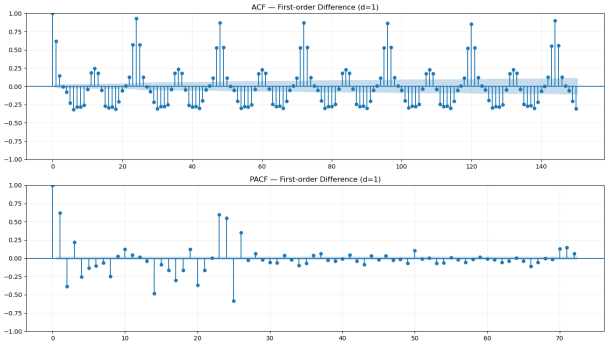


Figure 12: *ACF and PACF after first differencing. Slow decay at multiples of 24 argues for a seasonal component.*

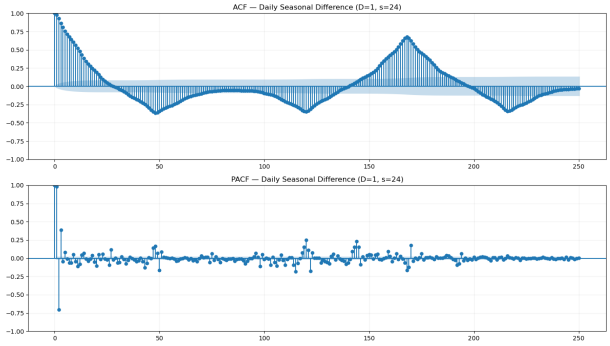


Figure 13: *ACF and PACF after daily seasonal differencing. The non-seasonal structure (lags 1–5) becomes more readable.*

For the ∇ family ($d = 1$, $D = 0$), the orders explored concentrate around $p \in \{2, 3\}$, $q \in \{1, 2\}$,

$P \in \{1,2\}$, $Q \in \{1,2\}$. For the ∇_{24} family ($d = 0$, $D = 1$, $s = 24$), the seasonal ACF rather suggests $Q = 0$ and a possibly long q, which is very costly in estimation time. A third family, $\nabla\nabla_{168}$, is also tried, with more parsimonious orders.

6.4 Estimation and information criterion

Each candidate is estimated by maximum likelihood. Models are compared by AIC,

$$\mathrm{AIC} = -2\log\widehat{\mathcal{L}} + 2k, \tag{6}$$

where k is the number of parameters and $\widehat{\mathcal{L}}$ the maximised likelihood. AIC penalises complexity, but guarantees neither numerical convergence nor residual whiteness. On this series, several rich specifications (in particular MA of order 5, 7 or 24) display a lower AIC *without converging*. They are discarded. The best converged compromise in the ∇ family is

$$\mathrm{SARIMA}(3,0,3) \times (0,1,0)_{24}, \tag{7}$$

with AIC = 136156 and a log-likelihood of −68071. This is the model, denoted Diff(1) for convenience in the tables — it combines daily seasonal integration and ARMA$(3,3)$ dynamics — that is retained for forecasting.

6.5 Residual diagnostics

A well-specified SARIMA must leave a residual indistinguishable from white noise. The temporal scatter, the residual ACF, and a global autocorrelation test by Ljung–Box are inspected. The statistic

$$Q_h = n(n+2)\sum_{k=1}^{h}\frac{\hat{\rho}_k^2}{n-k} \tag{8}$$

follows, under H_0 (all autocorrelations up to lag h are zero), a χ^2 law with h degrees of freedom, possibly adjusted for the number of parameters. A p-value above 5% is desired.

This is not the case. At the 1-hour horizon as at the 24-hour horizon, Ljung–Box rejects H_0 outright ($Q_1 = 394$, $Q_{24} = 591$, null p-values). Residuals remain autocorrelated. The model has captured part of the dynamics, not all. This is a warning carried into the forecasting stage: an honourable AIC is not an operational guarantee.

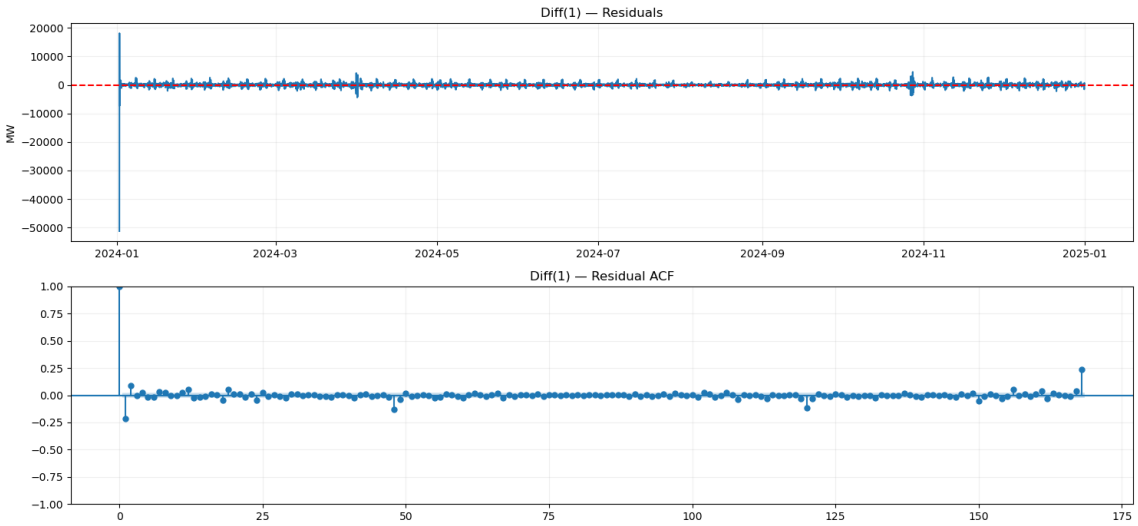


Figure 14: *Residuals of the retained SARIMA. Error clusters and a still significant ACF confirm that the dynamics are not exhausted.*

6.6 D+1 forecasts and Gaussian intervals

The retained model is then evaluated in the operational protocol, day after day, over the whole of 2025. At each origin, the 24 slots of $D+1$ and the 95% Gaussian forecast interval are produced,

$$\hat{y}_t \pm 1{,}96\,\widehat{\sigma}_t, \tag{9}$$

where $\widehat{\sigma}_t$ is the forecast standard deviation supplied by the model. Figure 15 gives a winter excerpt: the curve follows the daily rhythm, but the intervals are very wide, and some peaks escape it.

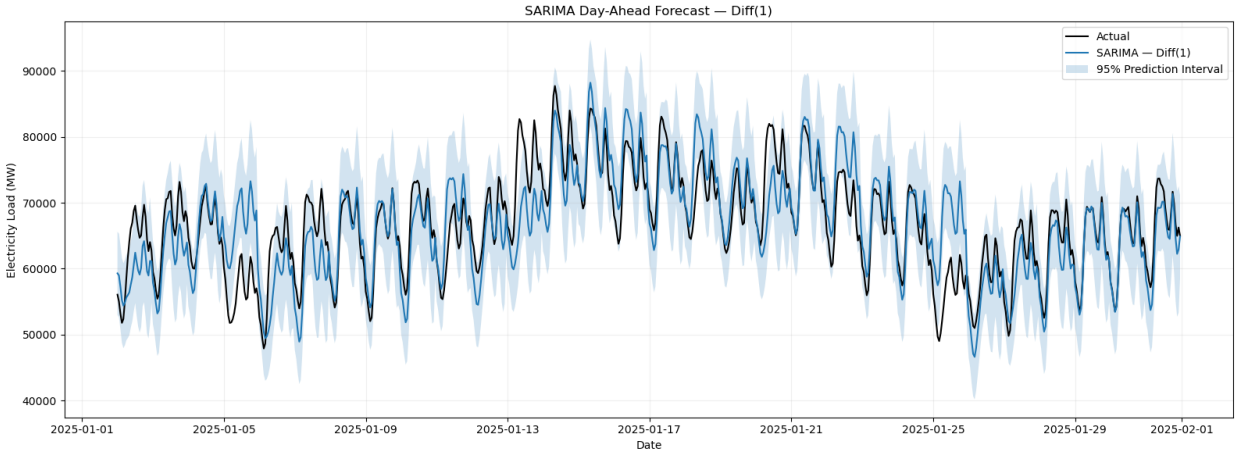


Figure 15: *SARIMA D+1 forecast, January 2025, with 95% Gaussian intervals. The model follows the cycle, at the cost of an uncertainty band more than 15 GW wide.*

On 8 735 comparable hours, SARIMA reaches a MAPE of 5.20%. This is better than the seasonal naive (6.41%), and clearly worse than RTE (2.81%). Empirical interval coverage is 92.96%, below the 95% target, for a mean width of 15 215 MW. In other words: too much error, too wide, and even so not quite well enough calibrated.

6.7 SARIMAX: adding heating and cooling degrees

The exploratory diagnosis suggested that temperature acts through thresholds. Two regressors are built from temperature lagged by 48 hours — the last weather information compatible with the 11:00 origin for all slots of D+1:

$$\mathrm{HDD}_t = \max(0,\, 15 - T_{t-48}), \qquad \mathrm{CDD}_t = \max(0,\, T_{t-48} - 22). \tag{10}$$

The model becomes a SARIMAX, that is a SARIMA on the residual of a regression

$$y_t = \beta_{\mathrm{HDD}}\,\mathrm{HDD}_t + \beta_{\mathrm{CDD}}\,\mathrm{CDD}_t + u_t, \tag{11}$$

where u_t follows the retained SARIMA dynamics. The estimated coefficients go in the right physical direction: $\hat{\beta}_{\mathrm{HDD}} = -21{,}4$ and $\hat{\beta}_{\mathrm{CDD}} = +51{,}4$. One cooling degree weighs more than one heating degree, which may surprise given winter peaks, but is explained by the fact that HDD is much more often non-zero, while CDD captures rarer and more marked episodes.

Operationally, however, the gain is nil. MAPE moves from 5.20% to 5.22%, coverage stays at 92.95%, width at 15 272 MW. Adding a *past* temperature lagged by 48 hours does not replace a next-day weather forecast. RTE very probably has that forecast. SARIMAX, in this protocol, does not.

6.8 Holt–Winters

Holt–Winters exponential smoothing offers a counterpoint. Where SARIMA models noise correlations, Holt–Winters explicitly smooths three states: a level ℓ_t, a trend b_t, a season s_t of period $m = 24$. In the damped additive version,

$$\ell_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + \phi b_{t-1}), \tag{12}$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)\phi b_{t-1}, \tag{13}$$

$$s_t = \gamma(y_t - \ell_t) + (1-\gamma)s_{t-m}, \tag{14}$$

and the forecast at horizon h is written $\hat{y}_{t+h} = \ell_t + (\sum_{i=1}^{h} \phi^i) b_t + s_{t+h-m}$. Four specifications are estimated on the training set, then updated at each validation origin, using only the available past. The best, HW_2 (damped additive trend, multiplicative seasonality of period 168), reaches 6.36% MAPE. The others, between 7.57% and 7.66%, are worse than the seasonal naive. Intervals, calibrated in a conformal way on the first half of 2025, cover too much (96 to 99%) because they are too wide: 20.5 to 21.3 GW.

6.9 Statistical models: summary

Table 6: *Statistical comparison on the common hours of 2025 (8 735 observations).*

Model	MAPE (%)	MAE (MW)	RMSE (MW)	Cov. 95 % (%)	Width (MW)
RTE D+1	**2,81**	**1395**	**1724**	—	—
SARIMA Diff(1)	5,20	2630	3807	92,96	15 215
SARIMAX (HDD+CDD)	5,22	2636	3811	92,95	15 272
Holt–Winters HW_2	6,36	3291	4334	97,32	20 541
Seasonal naive 168 h	6,41	3448	5136	—	—
Holt–Winters HW_4	7,57	3795	4977	96,65	21 317
Holt–Winters HW_3	7,60	3751	4950	96,11	20 705
Holt–Winters HW_1	7,66	3784	4981	95,90	20 632

Statistical models are not useless. SARIMA beats the seasonal naive. They describe linear seasonal dynamics that could be written, tested and diagnosed. But they fail on the two criteria that matter for an operator: point error and interval width. They capture neither calendar non-linearities (a long-weekend Monday is not an ordinary Monday) nor the information already contained in the RTE forecast. This double finding is precisely what justifies moving to LightGBM.

7 From statistical tables to LightGBM

LightGBM (Light Gradient Boosting Machine) is a decision-tree boosting algorithm, designed for tabular features. Boosting consists in successively adding weak trees, each correcting the errors of the previous one. Unlike SARIMA, LightGBM does not impose a linear form, a unique seasonality, or a Gaussian residual. It can learn interactions: a Saturday in August does not have the same effect as a Saturday in January; a load lag does not have the same weight at 19:00 and at 03:00. The price to pay is feature engineering. A tree sees seasonality only if it is supplied explicitly, through lags, hours and scores.

7.1 Gradient boosting as residual learning

The selected machine-learning approach does not directly model demand: it estimates the signed error of an already available day-ahead forecast. For hourly target y_t and published RTE forecast $\hat{y}_t^{\mathrm{RTE}}$, the learning target is

$$e_t = y_t - \hat{y}_t^{\mathrm{RTE}}, \qquad \widehat{y}_t^{\mathrm{final}} = \hat{y}_t^{\mathrm{RTE}} + F_M(\mathbf{x}_t). \tag{15}$$

The boosted predictor is an additive ensemble of regression trees f_m:

$$F_0(\mathbf{x}) = c, \qquad F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta f_m(\mathbf{x}), \tag{16}$$

$$r_{i,m} = -\left.\frac{\partial \ell(e_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right|_{F=F_{m-1}}, \qquad F_M(\mathbf{x}) = c + \eta \sum_{m=1}^{M} f_m(\mathbf{x}). \tag{17}$$

Here η is the shrinkage rate and $r_{i,m}$ is the negative loss gradient that steers each new tree. LightGBM uses first- and second-order gradient information for tree-leaf optimization. With squared-error loss, this amounts to fitting the errors remaining after the current ensemble. The learned quantity is in megawatts: a positive F_M increases the RTE forecast, while a negative value reduces it. Crucially, every element of $\mathbf{x}_t$ must be observable at the stated forecast origin.

Three architectures will be compared, all under the same protocol, the same 2019–2024 training, the same 2025 validation, and — at first — the same hyperparameters. Only the feature vector and the target change.

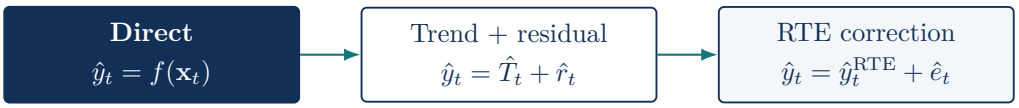


Figure 16: *The three LightGBM families, from the most "autonomous" to the most anchored in the existing operational forecast.*

7.2 Calendar feature engineering

The calendar is not a list of dummy indicators. We have seen that a day is described by its season, its holiday type, its position in the week and its hour. This operational knowledge is translated into scores, built in Paris time, according to heuristic rules — these are not estimated coefficients, they are priors that LightGBM is free to use, ignore or twist.

Table 7: *Calendar-score rules. Holidays are approximated by recurrent date ranges, not by official academic calendars year by year.*

Component	Condition	Score
Season	winter (Dec.–Feb.)	+2
	summer (Jun.–Aug.)	−2
	mild shoulder seasons	−1
	other	0
Holidays	normal activity	+2
	school holidays only	−1
	business holidays only	−2
	both	−3
Day	working day	+1
	weekend	−1
	extra business-slowdown period	−1
Hour	peak 05:00–08:00 or 17:00–19:00	+1
	other hours	−1

Two aggregates are derived from them. The raw score is the sum of the four components,

$$C_t = S_t + H_t + D_t + U_t. \tag{18}$$

The weighted score puts more weight on the intra-day cycle, then on the season and the day type,

$$C_t^w = \frac{4\,S_t + H_t + 2\,D_t + 8\,U_t}{15}. \tag{19}$$

This second indicator, denoted *calendar amplify ewa* in the experimental work, is the one that will survive in the best correction model.

7.3 Load and weather lags

For LightGBM to see the past, lags are supplied explicitly, always computed on the continuous UTC index, and always compatible with the 11:00 origin. The 48-hour lag y_{t-48} describes the recent level. The 168-hour lag y_{t-168} describes the same hour of the previous week. Temperature T_{t-48} and the degree-days

$$\mathrm{HDD}_t^{(48)} = \max(0, 15 - T_{t-48}), \qquad \mathrm{CDD}_t^{(48)} = \max(0, T_{t-48} - 22) \tag{20}$$

bring the available weather context. These five variables, plus the target hour, weekday and month, form the common base of the "direct" configurations.

7.4 RTE features and standardisation

When the target is error correction, two additional variables become central. The RTE forecast of the target slot, $\hat{y}_t^{\mathrm{RTE}}$, and the historical error

$$e_{t-48}^{\mathrm{RTE}} = y_{t-48} - \hat{y}_{t-48}^{\mathrm{RTE}}. \tag{21}$$

The first says what RTE thinks of the following day. The second says how RTE recently erred, at the same time of day. A standardisation (zero mean, unit variance) is optionally applied, fitted exclusively on 2019–2024, then frozen. No information from 2025 or 2026 enters the mean or the standard deviation.

Five feature sets are thus built, all on the same 65 710 hours, but with different columns: *detailed* (separate calendar scores), *aggregated* (raw sum), *weighted* (weighted sum), *rte* (raw RTE forecast and error), *rte_scaled* (same variables, standardised). A sixth set, *rte_calendar*, will later combine the RTE forecast, the past error, degree-days and the weighted score.

7.5 Shared training protocol

A single global regressor predicts all hours. Forecasts are not chained into one another (no recursive strategy): each slot of $D{+}1$ is forecast directly from its own vector $\mathbf{x}_t$. The loss is quadratic. Incomplete observations (start of the series, lags not yet defined) are dropped from training. Initial hyperparameters are shared: 1000 trees, learning rate 0.03, 15 leaves, depth 12, row subsampling (0.8) and column subsampling (0.9), unit ℓ_2 penalty. This configuration is not meant to be optimal: it isolates the effect of features and of the target. Optimisation will come afterwards, on the architectures that deserve it.

7.6 Direct forecasting

The three calendar representations — detailed, aggregated, weighted — are first trained to predict y_t itself. On the 2025 validation, they are almost equivalent, between 3.75% and 3.77% MAPE. The weighted version is very slightly ahead. All beat the seasonal naive. None reaches RTE (2.82%). Calendar aggregation, at this stage, is therefore not the decisive lever. The lever, if it exists, lies elsewhere: in how the problem is posed.

Table 8: *Direct LightGBM, 2025 validation (8 759 hours).*

Model	MAPE (%)	MAE (MW)	RMSE (MW)
RTE D+1	**2,82**	**1397**	**1726**
Weighted LightGBM	3,75	1976	2866
Aggregated LightGBM	3,76	1981	2867
Detailed LightGBM	3,77	1986	2882
Seasonal naive 168 h	6,43	3461	5146

7.7 Local trend and residual

The next idea is inherited from statistical decomposition. A local level $\hat{T}_{t|D}$ is estimated by exponential moving average, computed only on observations available before 11:00 on day D. LightGBM no longer learns y_t, but the residual

$$r_t = y_t - \hat{T}_{t|D}, \qquad \hat{y}_t = \hat{T}_{t|D} + \hat{r}_t. \tag{22}$$

The model no longer has to relearn the weekly level: it concentrates on the gap to that level. The three calendar representations are retrained on this target. The gain is clear, of the order of 0.3 MAPE point. The best, Trend + aggregated LightGBM, falls to 3.43%. We get closer to RTE, we still do not beat it.

7.8 Correcting the RTE forecast error

The target is then changed one last time. Instead of predicting load, one predicts the RTE error,

$$e_t = y_t - \hat{y}_t^{\mathrm{RTE}}, \qquad \hat{y}_t = \hat{y}_t^{\mathrm{RTE}} + \hat{e}_t^{\mathrm{LGBM}}. \tag{23}$$

This is the core of the project. LightGBM no longer has to reinvent a national forecast: it learns recurrent biases — a poorly anticipated public-holiday Monday, an underestimated evening peak, an error that repeats 48 hours apart — and subtracts them.

Three configurations are compared. The base version uses only the RTE forecast, its past error and the minimal calendar (hour, day, month). The standardised version identifies the same variables after a z-score. The calendar version adds degree-days and the weighted score. All three, from the outset, go below RTE.

Table 9: *All LightGBM architectures, 2025 validation, same hours.*

Model	MAPE (%)	MAE (MW)	RMSE (MW)
RTE + LGBM (calendar)	**1,61**	**823**	1086
RTE + LGBM (standardised)	1,61	822	**1083**
RTE + LGBM (base)	1,61	823	1084
RTE D+1	2,82	1397	1726
Trend + LGBM (aggregated)	3,43	1794	2506
Trend + LGBM (detailed)	3,44	1800	2516
Trend + LGBM (weighted)	3,46	1812	2530
Direct LGBM (weighted)	3,75	1976	2866
Seasonal naive 168 h	6,43	3461	5146

The jump is of a different nature from the previous ones. One no longer gains a few tenths, one gains more than one MAPE point, and one goes *below* the operational benchmark. The three RTE variants are equivalent at this stage, a sign that the bulk of the information is already in the forecast and in its recent error. The richer calendar version is nevertheless retained for optimisation, alongside the best direct model (weighted) and the best trend hybrid (aggregated).

Assumed limitation

This whole family assumes that $\hat{y}_t^{\mathrm{RTE}}$ is known at 11:00 on day D for each of the 24 slots of D+1. If, in operational reality, RTE publishes later, or publishes only a partial curve, the correction model is no longer deployable as such. This is the main reservation of the study, and it does not concern the direct models nor the statistical models.

7.9 Hyperparameter search

Optimisation is chronological, never random in time. Each candidate is trained on 2019–2023 and scored on the valid hours of 2024, according to the MAPE of *reconstructed* load (residual + trend, or residual + RTE forecast). Fifteen random draws per architecture, i.e. 45 trainings, explore the number of leaves, depth, number of trees, learning rate, ℓ_2 penalty and minimum leaf size.

The best set for RTE + calendar LightGBM brings the 2024 MAPE down to 1.678%, with 500 trees, 31 leaves, unconstrained depth, 200 minimum observations per leaf, $\lambda = 5$ and a step of 0.03. The trend hybrid improves to 3.30%, the direct model to 3.88%. The three winners are then retrained on the whole of 2019–2024, then evaluated walk-forward on 2025.

Table 10: *Optimised architectures, 2025 validation.*

Model	MAPE (%)	MAE (MW)	RMSE (MW)
RTE + calendar LGBM [tuned]	**1,60**	**817**	**1080**
RTE D+1	2,82	1397	1726
Trend + aggregated LGBM [tuned]	3,41	1786	2491
Direct weighted LGBM [tuned]	3,77	1994	2913
Seasonal naive 168 h	6,43	3461	5146

Tuning confirms the architectural diagnosis. Optimising the direct model or the trend hybrid does not let them catch RTE. Optimising error correction widens the gap a little further, from 1.61% to 1.60%. It is not the tuning that makes the result, it is the choice of correcting RTE rather than competing with it.

7.10 Global comparison on validation

Finally, the best representatives of each family are gathered on the 4 416 common hours of July–December 2025 — the window that will also serve to check conformal intervals.

Table 11: *Global comparison, July–December 2025 (4 416 common hours).*

Model	MAPE (%)	MAE (MW)	RMSE (MW)	Cov. 95 % (%)	Width (MW)
RTE + LGBM [tuned]	**1,57**	**779**	**1040**	96,69	4 739
RTE D+1	2,73	1330	1672	—	—
Trend + LGBM [tuned]	2,98	1536	2205	—	—
Direct LGBM [tuned]	3,29	1718	2654	—	—
SARIMA Diff(1)	4,95	2440	3638	93,39	15 216
SARIMAX	4,97	2446	3640	93,39	15 273
Naive 168 h	5,17	2768	4588	—	—
Holt–Winters HW_2	5,68	2820	3692	99,05	20 541

The ranking is unambiguous. RTE + LightGBM correction reduces relative error by 42.5% versus RTE on this window, with intervals already narrower than those of SARIMA, and coverage a little above 95%. This is the model, and it alone, that is taken to the 2026 test, still intact.

8 Final evaluation on 2026

The 2026 test is not a second validation. Hyperparameters are frozen, the model is the one trained on 2019–2024, interval margins are calibrated on January–June 2025, and evaluation covers January–June 2026, i.e. 4 343 valid hours. None of these hours was used to choose the model.

8.1 Hourly conformal prediction intervals

SARIMA Gaussian intervals were both too wide and poorly calibrated, because residuals were neither Gaussian nor white. Another, non-parametric path is adopted: conformal prediction by absolute-error quantiles, stratified by civil hour. The idea is simple. On a calibration period, past errors are observed. From them one infers, for each hour h, a margin $\hat{q}_h$ such that the interval $\hat{y}_t \pm \hat{q}_h$ contains, on average, at least 95% of future realisations, without a normality assumption.

Let $\alpha = 0{,}05$ be the target risk, and let $\mathcal{C}_h$ be the set of absolute errors $|y_t - \hat{y}_t|$ observed, for civil hour h, between January and June 2025. These errors are sorted, $e_{(1)} \leq \cdots \leq e_{(n_h)}$, and the conformal quantile is taken

$$\hat{q}_h = e_{(k_h)}, \qquad k_h = \lceil (n_h + 1)(1 - \alpha) \rceil. \tag{24}$$

The rank k_h is not the usual empirical $1 - \alpha$ quantile: the term $(n_h + 1)$ is the conformal correction that guarantees a marginal coverage at least equal to $1 - \alpha$ under exchangeability. The forecast interval for a 2026 observation falling at hour h is then

$$\left[\hat{y}_t - \hat{q}_h,\ \hat{y}_t + \hat{q}_h\right]. \tag{25}$$

Margins are computed separately for RTE and for LightGBM, then frozen. They are not recalibrated on 2026. The exchangeability assumption for errors, within each hour, is only imperfectly verified (residuals remain autocorrelated). Despite that, empirical coverage on 2026 sits near the 95% target, and closer to that target for LightGBM than for RTE.

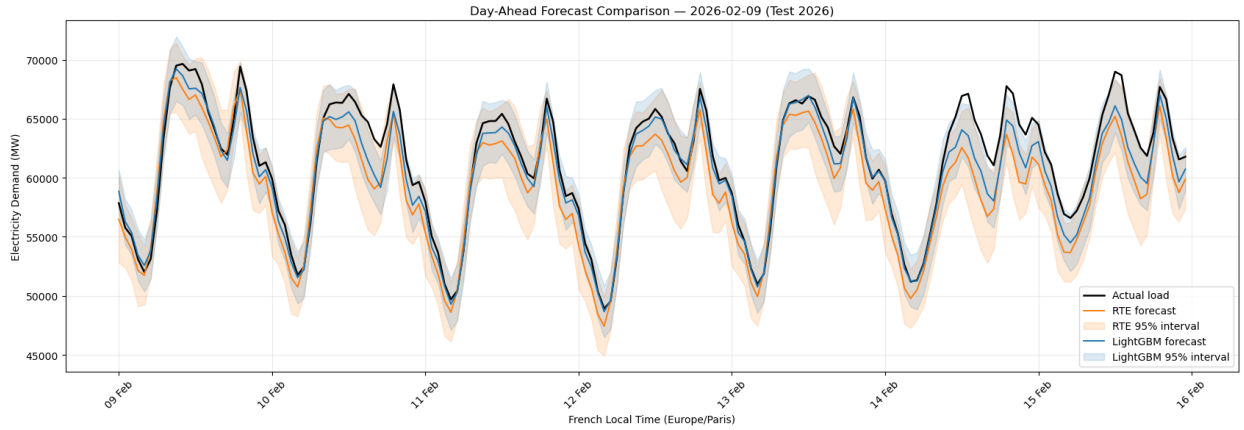


Figure 17: *Week of 9 February 2026, held out throughout development. In black, realised load. In orange, RTE and its interval. In blue, corrected LightGBM and its interval, visibly narrower, and closer to the black curve, in particular on evening peaks.*

Over the whole half-year, MAPE moves from 2.99% to 1.77% (−40,8% relative). Mean interval width moves from 6 233 MW to 4 625 MW (−25,8%). The corrected model is not only more accurate on average: it is more *precise* in its admission of uncertainty.

8.2 Residual diagnostics

The point error $e_t = y_t - \hat{y}_t$ is the raw material of everything that remains to be improved. Figure 18 shows it for both models, in level and in ACF.

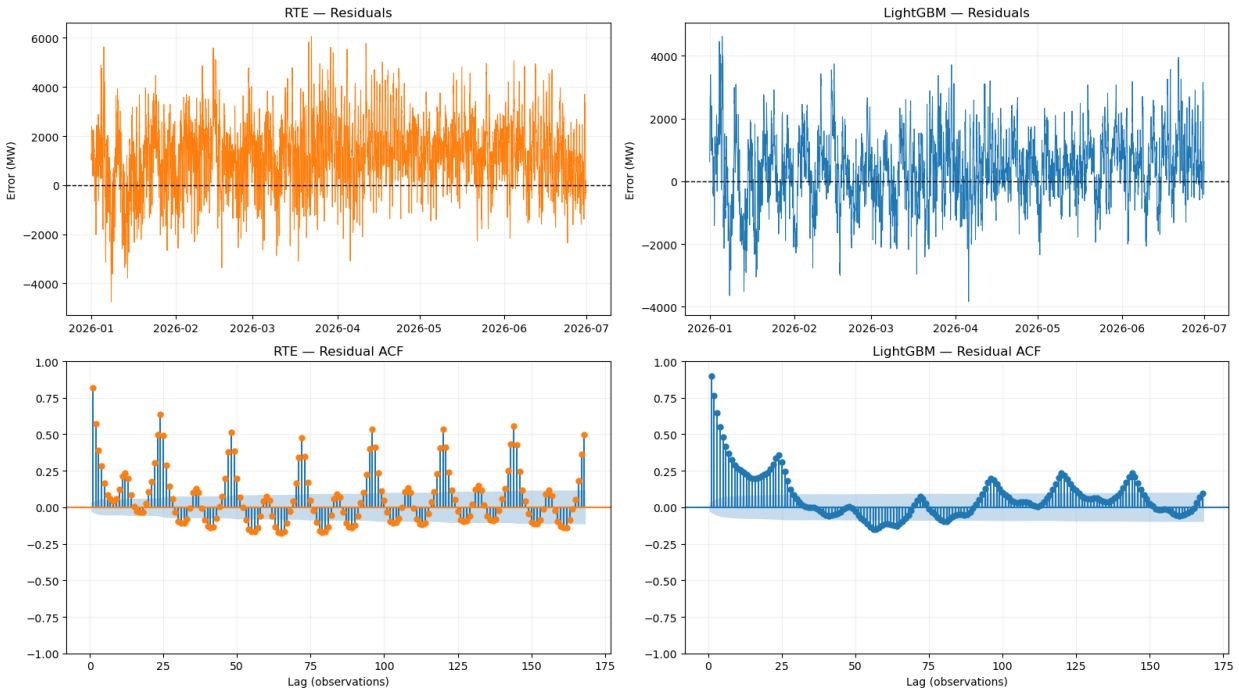


Figure 18: *Top: 2026 residuals. LightGBM strongly tightens the amplitude, around zero. Bottom: ACF. RTE exhibits sharp peaks at multiples of 24 hours. LightGBM attenuates these peaks, but leaves a strong short-term autocorrelation.*

LightGBM clearly reduces bias and amplitude. RTE residuals often oscillate between −4 and +6 GW, with a positive drift (chronic underestimation). LightGBM residuals stay more within ±3 GW. However, neither is white noise. The ACF of RTE is a daily comb: lags 24, 48, 72, …, 168. That of LightGBM is smoother, with a slow decay on the first lags, then more modest bumps at daily periodicities.

The Ljung–Box test, at lags 24, 48 and 168, rejects H_0 for both models, with p-values below 10^{-3}. The statistics are even higher in the short term for LightGBM ($Q_{24} = 16538$ against 9705 for RTE), which confirms the visual diagnosis: the correction has absorbed a large part of RTE's daily error, but it has left — or created — a very short-term serial dependence.

Table 12: *Ljung–Box test on the 2026 test residuals. H_0: no autocorrelation up to the indicated lag.*

Model	Lag	Q	p-value	Decision
RTE	24	9705	$< 10^{-3}$	reject
RTE	48	13 826	$< 10^{-3}$	reject
RTE	168	32 945	$< 10^{-3}$	reject
LightGBM	24	16 538	$< 10^{-3}$	reject
LightGBM	48	17 565	$< 10^{-3}$	reject
LightGBM	168	22 628	$< 10^{-3}$	reject

Overlaying the two ACFs (Figure 19) clarifies what the correction has actually changed. LightGBM reduces RTE's 24 h autocorrelation peaks, hence part of the systematic daily error. By contrast, autocorrelation at the very first lags remains high: with the features it has, the model has

not captured the correlation between an error at 18:00 and the error at 19:00. A natural track, not explored here, would be to add an ARMA post-processing on LightGBM residuals, using only residuals available before 11:00, so as not to recreate leakage.

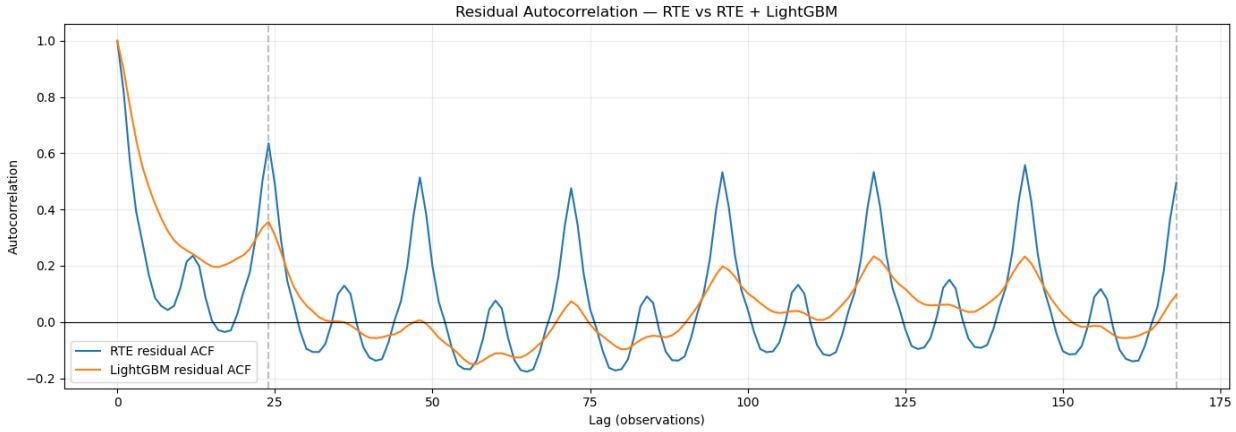


Figure 19: *Overlaid ACFs. Blue: RTE, daily comb. Orange: LightGBM, attenuated daily peaks, strengthened short memory.*

8.3 When the models fail

MAPE summarises mean error, but not accidents. Two types of failure are defined. An *interval failure* occurs when y_t leaves the 95% conformal interval. A *point failure* occurs when the relative error exceeds 5%,

$$\mathrm{APE}_t = 100 \left| \frac{y_t - \hat{y}_t}{y_t} \right| > 5. \tag{26}$$

Figure 20 returns to the week of 9 February. Red points (outside the interval) and orange points (APE $> 5\%$) are clearly more numerous for RTE. For LightGBM, remaining interval failures often coincide with zones already difficult for RTE — typically a poorly anticipated peak. By contrast, several evenings where RTE leaves its interval, or exceeds 5% error, are recovered by the correction. LightGBM does not copy RTE errors: it erases part of them, and stumbles with RTE on the most atypical episodes.

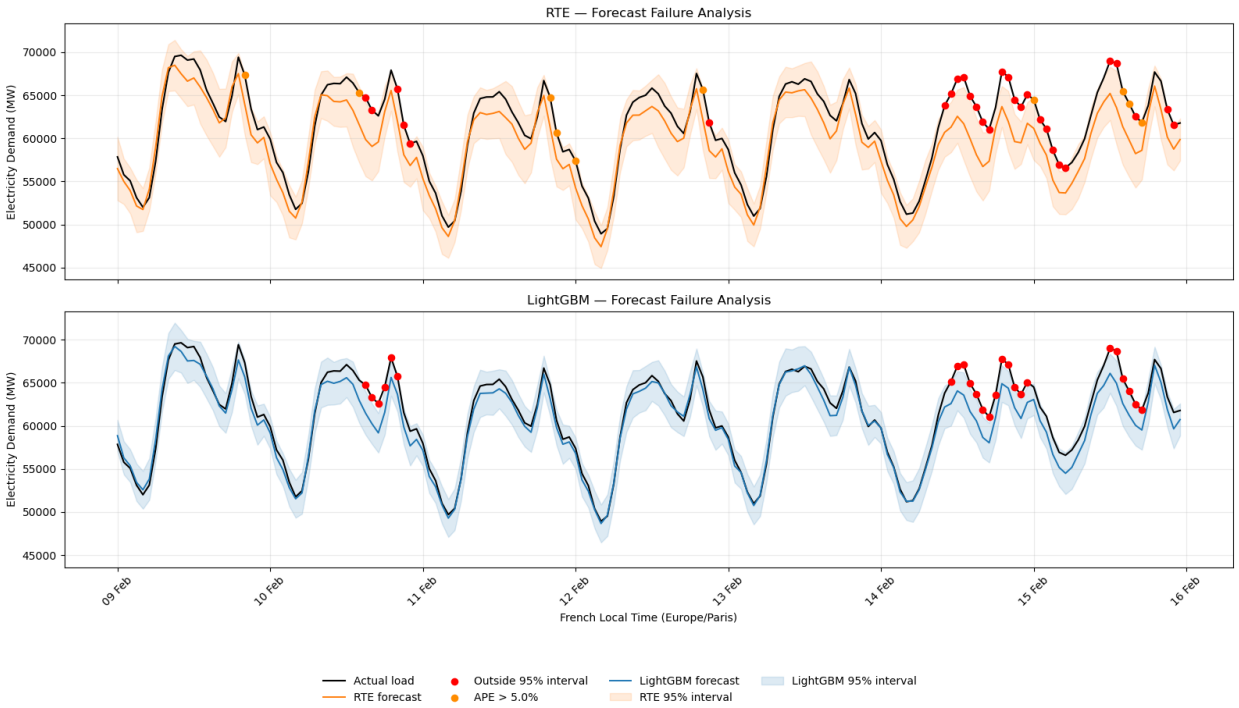


Figure 20: *Failure analysis, week of 9 February 2026. Red: observation outside the 95% interval. Orange: APE above 5%. LightGBM fails mainly where RTE already fails, and avoids a substantial fraction of RTE's additional failures.*

To judge the regularity of this behaviour, APE over the whole half-year is normalised by the 5% threshold,

$$\tilde{e}_t = \frac{\mathrm{APE}_t}{5}. \tag{27}$$

A value above 1 is a point failure. Figure 21 shows that the orange bars (RTE) frequently exceed this threshold, especially in spring, while the blue bars (LightGBM) remain, in their large majority, below it. The improvement is not concentrated on a few lucky weeks: it holds over six months never seen before.

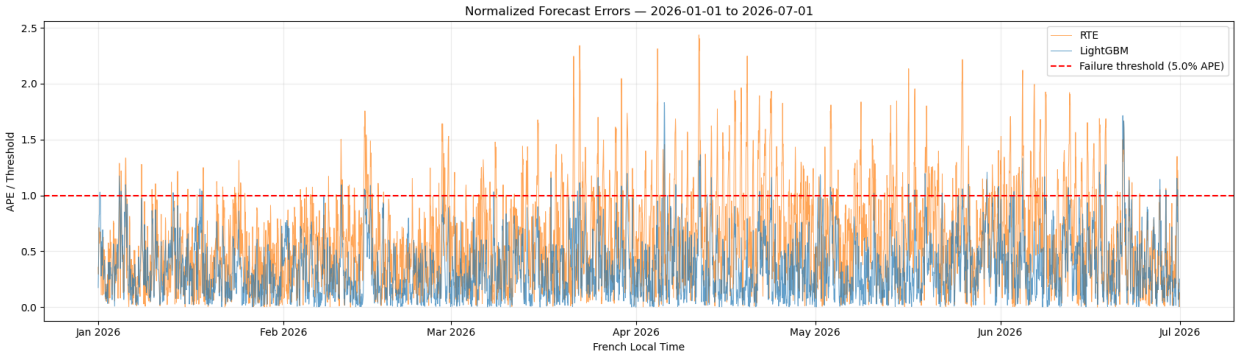


Figure 21: *Relative error normalised by the 5% threshold, January–June 2026. Above 1, the forecast is counted as a point failure. LightGBM stays there much more rarely than RTE.*

8.4 Consolidated table

Table 13: *Comparison of RTE and RTE + LightGBM on the 2026 test set (4 343 hours). For MAPE, width and failure rates, a negative relative change is an improvement. For coverage, a positive change is an improvement (one gets closer to the 95% target).*

Metric	RTE	LightGBM	Δ abs.	Δ rel.
MAPE (%)	2,99	1,77	−1,22	−40,75 %
95% coverage (%)	93,51	95,12	+1,61	+1,72 %
Mean width (MW)	6 233	4 625	−1608	−25,80 %
Interval failures (%)	6,49	4,88	−1,61	−24,82 %
Point failures, APE > 5% (%)	16,33	2,21	−14,11	−86,46 %

On the 4 343 hours of the 2026 test set, RTE + LightGBM clearly outperforms the original RTE forecasts, both in accuracy and in reliability. First, it reduces mean forecast error (MAPE) by **40.75%**, from 2.99% to 1.77%. The result is even more relevant for uncertainty management: at the **same 95% coverage target**, the model produces prediction intervals that are **25.80% narrower**, while achieving better empirical coverage (95.12% versus 93.51%). In other words, one forecasts more accurately, with less displayed uncertainty, without sacrificing interval reliability. The same improvement also reduces interval failures by **24.82%** and, above all, errors above the 5% threshold by **86.46%**. These results show the interest of the approach: rather than rebuilding an electricity forecasting model from scratch, one exploits the quality of RTE forecasts and learns to correct their errors. Performance on data never used for training confirms that this strategy works over the period studied.

These figures do not say that the model generalises beyond June 2026, nor that it would remain superior if RTE changed its internal model. They also do not say that residuals are white noise: remaining autocorrelation, established by Ljung–Box, remains a statistical limit and a track for improvement.

9 Outlook: a multi-resolution Transformer

The diagnostics of the previous section leave an open question. LightGBM, as a tabular model, sees each hour as a row, equipped with a few explicit lags. It does not traverse a sequence. It therefore cannot, except by multiplying lags, learn a long, multi-scale dependence of the type "last week's shape modulates this evening's peak". Neural networks built for time series, and in particular Transformers with *patches*, are made for that. This section presents the principles of the *Multi-resolution Time-Series Transformer* (MTST) of Zhang, Ma, Pal, Zhang and Coates (AISTATS 2024), then the adaptation envisaged for French D+1 forecasting. The architecture is described first, followed by the training and evaluation subsequently carried out in the project notebook.

9.1 Why a Transformer, and why multi-resolution?

The first time-series Transformers treated each instant as a token. Attention then had to link 168 hours to 168 hours, without structure, and proved surprisingly insensitive to temporal order — to the point that simple linear models often beat them. The decisive idea, taken up by PatchTST and Crossformer, is to group consecutive instants into *patches*, and to make each patch a token. Attention then works at the scale of windows, not of instants.

Zhang et al.'s observation is that a single patch has a size, hence a frequency. A short patch (high resolution) sees transients, peaks, local noise. A long patch (low resolution) sees the trend, the week, the season. MTST refuses to choose. Each layer of the network has several branches, each tokenising the same series at a different patch size, applying its own attention mechanism, then fusing the representations.

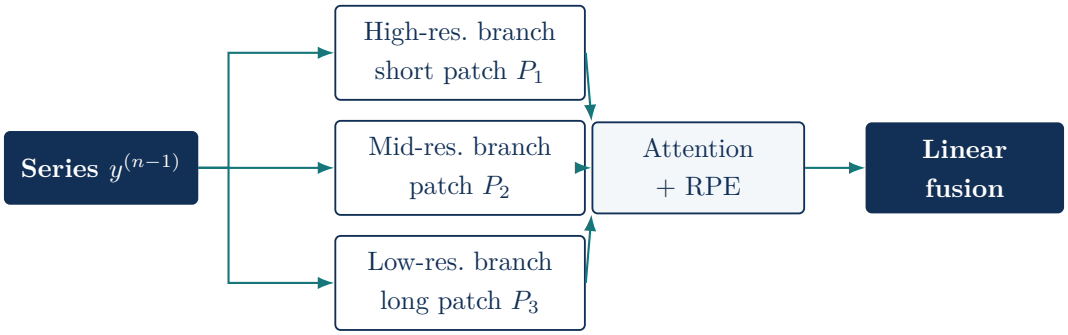


Figure 22: *Principle diagram of an MTST layer, after Zhang et al. (2024). Several tokenisers, several attentions, one fusion. Layers are stacked, the last producing the forecast horizon.*

9.2 Problem formulation

As in the paper, we place ourselves in a long-term multivariate forecasting setting, before specialising it. Let $\mathbf{x}_t \in \mathbb{R}^M$ be the vector of the M variables at instant t. From a history window of length L, one wants to forecast T future instants. For an arbitrary offset t_0,

$$\mathbf{x}_{t_0+1:t_0+L} \longmapsto \widehat{\mathbf{x}}_{t_0+L+1:t_0+L+T}. \tag{28}$$

Training is done on a set $\mathcal{D}_{\text{trn}}$ of examples $(\mathbf{x}^{(k)}_{1:L}, \mathbf{x}^{(k)}_{L+1:L+T})$ extracted by windowing. The paper's metrics are mean squared error and mean absolute error, normalised by the number of variables, horizons and test examples:

$$\text{MSE} = \frac{1}{MT\,|\mathcal{D}_{\text{test}}|} \sum_{k \in \mathcal{D}_{\text{test}}} \sum_{t=L+1}^{L+T} \left\| \mathbf{x}_t^{(k)} - \widehat{\mathbf{x}}_t^{(k)} \right\|_2^2, \tag{29}$$

$$\text{MAE} = \frac{1}{MT\,|\mathcal{D}_{\text{test}}|} \sum_{k \in \mathcal{D}_{\text{test}}} \sum_{t=L+1}^{L+T} \left\| \mathbf{x}_t^{(k)} - \widehat{\mathbf{x}}_t^{(k)} \right\|_1. \tag{30}$$

In practice MTST treats each variable separately (*channel independence*), then recomposes the multivariate forecast. Nothing conceptually forbids extending the mechanism to architectures that mix variables, of the Crossformer type.

9.3 Branch-specific tokenization

Let $y^{(n-1)} \in \mathbb{R}^{d_{n-1}}$ denote the univariate representation at the input of layer n, with $y^{(0)} = (x_1, \dots, x_L)^\top$ and, at the output of the last layer, $y^{(N)} = (\hat{x}_{L+1}, \dots, \hat{x}_{L+T})^\top$. On branch b of this layer, one takes a patch size P_b and a stride S_b. The tokeniser $\mathcal{T}_b$ splits $y^{(n-1)}$ into $J_b = \lceil (d_{n-1} - P_b)/S_b \rceil + 1$ possibly overlapping patches,

$$\tilde{y}_j^{(b,n)} = y^{(n-1)}_{\left[(j-1)S_b+1:(j-1)S_b+P_b\right]}, \qquad j = 1, \dots, J_b. \tag{31}$$

If the length is not divisible, one completes with *padding* at the end of the series. A small P_b (hence a large J_b) produces a high-resolution map, sensitive to local patterns. A large P_b produces a low-resolution map, sensitive to long seasons. That is the whole principle: one does not downsample the series, one changes the *token size*.

9.4 Attention and relative positional encoding

Each branch then applies multi-head attention on its own tokens. Let $\tilde{\mathbf{Y}} = (\tilde{y}_1, \dots, \tilde{y}_J)^\top$ be the patches of a branch. Attention scores between tokens i and j are written

$$\alpha_{ij} = \mathrm{Softmax}_j \left(\frac{(W_Q \tilde{y}_i)^\top (W_K \tilde{y}_j)}{\sqrt{D}} + w_{\mathrm{pos}}^\top p_{ij} \right), \qquad \tilde{z}_i = \sum_{j=1}^{J} \alpha_{ij} \, W_V \tilde{y}_j. \tag{32}$$

$W_Q, W_K, W_V \in \mathbb{R}^{D \times P}$ are learned projections, and one takes $D = P$ as in the original Transformer. The term p_{ij} is the *relative* positional encoding (RPE). This is a deliberate choice. An absolute encoding attaches to each token its place in the window. A relative encoding attaches to each *pair* the signed distance between tokens. For a periodic series, it is this distance that counts: the patch from 24 hours ago resembles the present patch, wherever in the window one stands.

Concretely,

$$p_{ij} = \mathrm{sign}(i - j) \, \mathrm{PE}(|i - j|), \tag{33}$$

where $\mathrm{PE} : \mathbb{N} \to \mathbb{R}^{D_{\mathrm{pos}}}$ is the classical sinusoidal encoding,

$$\mathrm{PE}_{2t}(i) = \sin\!\left(\frac{i}{10000^{2t/D_{\mathrm{pos}}}} \right), \qquad \mathrm{PE}_{2t+1}(i) = \cos\!\left(\frac{i}{10000^{2t/D_{\mathrm{pos}}}} \right). \tag{34}$$

Zhang et al. show, on a synthetic periodic example, that this relative attention recovers repeated patterns where absolute attention disperses. After attention, a standard block — residual connection, normalisation, feed-forward network — produces the branch representation.

9.5 Fusing branch representations

The B_n branches of a layer do not have the same number of tokens, nor the same patch size. To make them communicate, each matrix $\tilde{Z}^{(b,n)} \in \mathbb{R}^{J_b \times P_b}$ is flattened into a vector, concatenated, and linearly projected to the layer output dimension d_n:

$$z^{(n)} = \left[\mathrm{Flatten}(\tilde{Z}^{(1,n)}), \; \dots, \; \mathrm{Flatten}(\tilde{Z}^{(B_n,n)}) \right]^\top, \qquad y^{(n)} = W^{(n)} z^{(n)} + b^{(n)}. \tag{35}$$

An MTST layer therefore reduces to

$$y^{(n)} = \mathrm{Fuse}\Big(\mathrm{Attn}(\mathcal{T}_1(y^{(n-1)})),\ \ldots,\ \mathrm{Attn}(\mathcal{T}_{B_n}(y^{(n-1)}))\Big). \tag{36}$$

Stacking N such layers lets the network recompose, stage after stage, increasingly abstract patterns, always read at several scales at once.

From parallel resolutions to an hourly residual forecast

At MTST layer n, each branch b forms J_b overlapping patches of length P_b, projects them to tokens, and applies attention whose scores contain a sinusoidal *relative-distance* term. Each branch output $Z_b^{(n)}$ is flattened; the branch vectors are concatenated and linearly fused:

$$\mathbf{z}^{(n)} = \mathrm{Concat}_{b=1}^{B}\Big(\mathrm{vec}(Z_b^{(n)})\Big), \qquad \mathbf{y}^{(n)} = W_n\mathbf{z}^{(n)} + \mathbf{b}_n. \tag{37}$$

The fused representation enters the following layer, rather than being combined only at the final head. In our RTE-correction setting, the forecast head returns a vector of hourly errors, and demand is reconstructed elementwise:

$$\widehat{\mathbf{y}}_{D+1}^{\mathrm{MTST}} = \widehat{\mathbf{y}}_{D+1}^{\mathrm{RTE}} + \widehat{\mathbf{e}}_{D+1}^{\mathrm{MTST}}. \tag{38}$$

9.6 Adaptation to French D+1 forecasting

The architecture above is put to the same use as LightGBM: correcting the public RTE forecast, hour by hour, for the 24 civil slots of day D+1. The model does not rebuild demand. It predicts the residual

$$e_t = y_t - \hat{y}_t^{\mathrm{RTE}},$$

and reconstructed demand is $\hat{y}_t = \hat{y}_t^{\mathrm{RTE}} + \hat{e}_t$. The implementation trained and evaluated below is released as a Python package on PyPI, and the model is also deposited on Hugging Face.

The eight inputs are those of the `rte_calendar` specification already selected for LightGBM: target hour, day of the week, month, the RTE forecast available at the origin, the RTE error lagged by 48 hours, heating and cooling degree-days at 48 hours, and the amplified calendar score. The target is the residual. A `StandardScaler` is fit on the training set only, separately for the features and for the target, then applied to validation and test. The resulting shapes are (52,607, 10) in training, (8,760, 10) in validation and (4,343, 10) in test. Training runs on Apple's GPU (MPS).

The training unit is not a sliding window of 336 hours. It is the French civil day. Each example has shape (24, 8) as input, (24) as target (the standardised residuals of the 24 hours) and (24) as a validity mask. At the autumn clock change, the last occurrence of hour 2 is kept. A day enters the set only if at least one slot is valid. This yields 2,190 training days, 365 validation days and 181 test days. Missing or non-existent slots are excluded from the loss by the mask, as in the rest of the project.

Three branches read that day at steps of 1 h, 2 h and 4 h, hence 24, 12 and 6 tokens. The head emits the 24 residuals directly. The initial loss is masked mean squared error. The optimiser is Adam, with learning rate $5 \cdot 10^{-4}$, weight decay 10^{-5}, 30 epochs and batches of size 8, without shuffling, so that chronological order is kept. Training mean squared error stays between 0.76 and

0.83. Training takes 6.02 minutes.

9.7 First result on 2025

On the 2025 validation set, after inverting the scaling and reconstructing demand, the model cuts RTE's error by about 35% in MAE.

Table 14: *2025 validation, reconstructed demand. First training run, masked squared loss.*

Model	MAE (MW)	RMSE (MW)	MAPE (%)	R^2
RTE	1,396.6	1,725.8	2.815	0.974
MTST	902.5	1,175.7	1.791	0.988

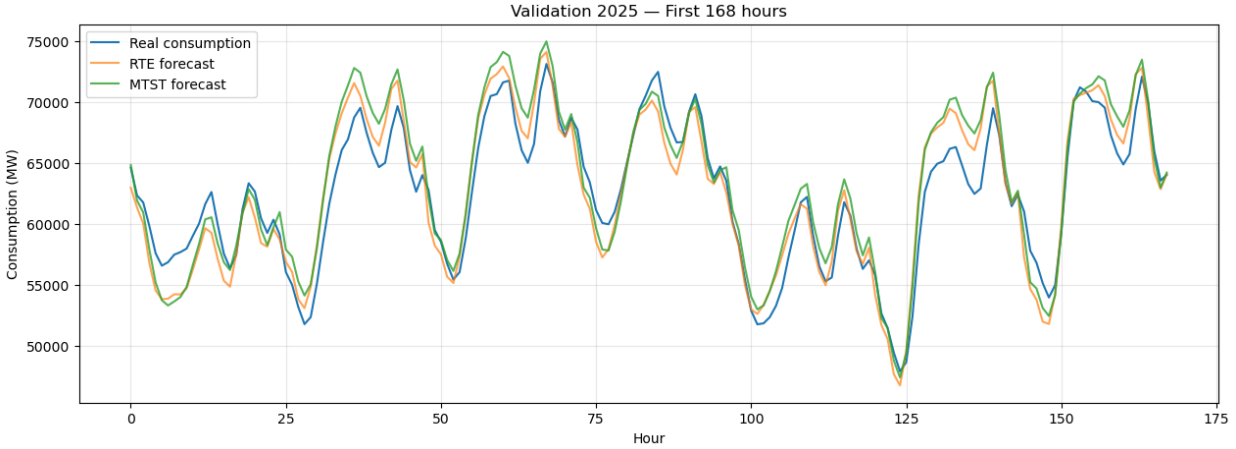


Figure 23: *2025 validation, first 168 hours. The MTST correction follows realised consumption and tightens the gap left by the RTE forecast, without erasing the daily profile.*

ADF and KPSS tests on the residuals of this validation, raw and then first-differenced, are consistent with stationarity: ADF rejects a unit root, and KPSS does not contradict that conclusion at the usual level, for both RTE and MTST. This check concerns the first validation run. It does not replace the conformal evaluation on the test set.

The 95% intervals are then built by hourly conformal quantiles, with the same rank rule as for LightGBM. Calibration uses the 2025 hours before 1 July. Evaluation is the second half of the year.

Table 15: *Second half of 2025. Conformal calibration before 1 July, evaluation from July.*

Indicator	RTE	MTST
MAE (MW)	1,329.6	862.7
RMSE (MW)	1,671.7	1,124.8
MAPE (%)	2.731	1.769
R^2	0.969	0.986
95% coverage (%)	95.95	96.97
Mean width (MW)	6,232.8	4,999.8
Interval failures (%)	4.05	3.03
APE $>$ 5% (%)	13.09	3.35

At coverage at least equal to the target, MTST reduces MAPE from 2.73% to 1.77%, shrinks

mean width by about 20%, and brings the share of errors above 5% from 13.1% to 3.4%. This gain is obtained before any hyperparameter search.

9.8 Tuning and retraining

Two Optuna searches, of 15 trials each, use the last 730 training days, split chronologically into 80% for fitting and 20% for scoring. Each trial runs for 10 epochs. Weak trials are stopped by median pruning. The search space is the embedding dimension (32, 64 or 128), the number of heads (4, 8 or 16), the number of layers (1 to 3), dropout between 0.1 and 0.3, the learning rate on a log scale from 10^{-4} to 10^{-3}, and the batch size (8, 16 or 32). Resolutions 1, 2 and 4 stay fixed.

The first search minimises residual MAE, in megawatts. The best trial reaches 787.41 MW, with embedding 128, 8 heads, 2 layers, dropout 0.289, learning rate $1.64 \cdot 10^{-4}$ and batches of 16. The search takes 2.65 minutes.

The second search minimises MAPE of reconstructed consumption, the project's primary metric. The best trial reaches 1.541%, with embedding 128, 16 heads, a single layer, dropout 0.185, learning rate $2.68 \cdot 10^{-4}$ and batches of 16. The search takes 5.82 minutes. This second configuration is the one that is kept.

The selected model is then retrained on the full 2019–2024 set for 50 epochs, with a masked MAPE loss computed on reconstructed demand in megawatts. The mean squared error of the standardised residuals, logged at each epoch, falls from 0.804 to 0.698. Training takes 8.88 minutes.

9.9 2026 test set

Forecasts are recomputed on 2025, which is the calibration set (8,759 hours), then on January–June 2026 (4,343 hours). Conformal margins are the hourly quantiles of absolute errors in 2025, at level $\alpha = 0.05$, with rank $\lceil (n+1)(1-\alpha) \rceil$. They are frozen before 2026 is touched.

Table 16: *Test set, January–June 2026.*

Indicator	RTE	MTST
MAE (MW)	1,334.2	808.7
RMSE (MW)	1,676.4	1,072.1
MAPE (%)	2.740	1.646
R^2	0.969	0.987
95% coverage (%)	95.09	96.30
Mean width (MW)	6,690.0	4,830.9
Interval failures (%)	4.91	3.70
APE > 5% (%)	13.20	2.65

On this test set, MTST reduces MAPE from 2.74% to 1.65%, a drop of 39.9%. MAE falls from 1,334 to 809 MW and RMSE from 1,676 to 1,072 MW. Mean interval width falls from 6,690 to 4,831 MW, 27.8% less, while empirical coverage stays above the target: 96.30% for MTST, against 95.09% for RTE. Interval failures fall from 4.91% to 3.70%. The share of hours whose relative error exceeds 5% falls from 13.20% to 2.65%, a drop of 79.9%. R^2 rises from 0.969 to 0.987. Accuracy and interval tightness both improve, and displayed coverage does not fall back below the target.

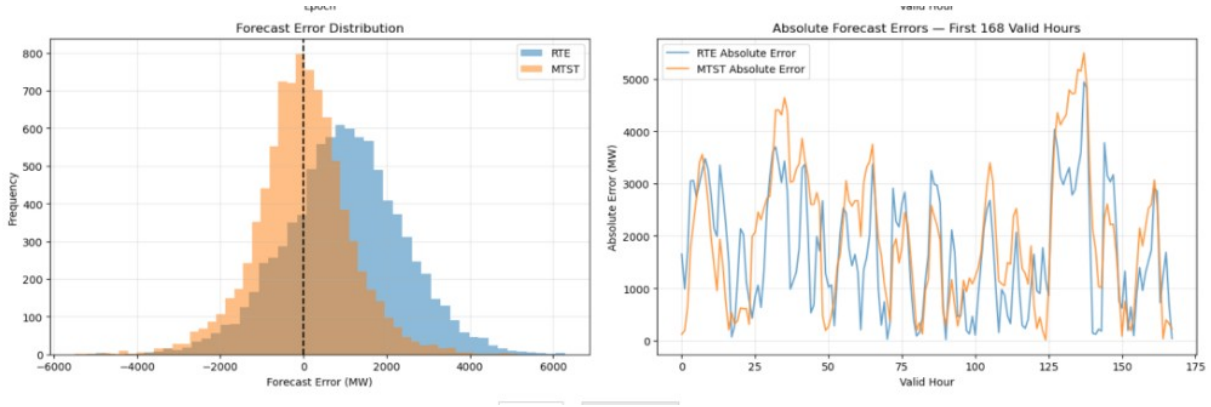


Figure 24: *Forecast diagnostic after training. Left: the distribution of signed errors. MTST is concentrated around zero, while RTE is shifted toward positive errors, that is, an under-prediction of demand. Right: absolute error over the first 168 valid hours. MTST is usually below RTE, with a few spikes where the correction exceeds the public error.*

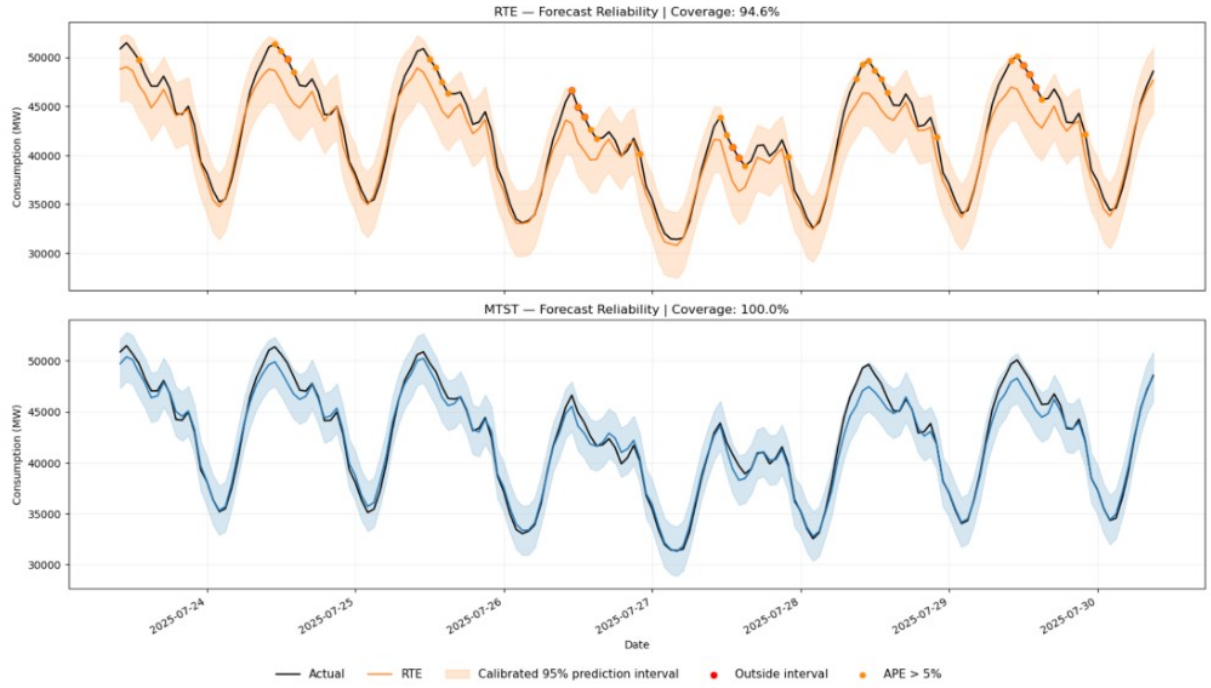


Figure 25: *Week of 24–30 July 2025. Top: RTE, with 94.6% coverage over the week and several hours above the 5% relative-error threshold (orange points). Bottom: MTST, with 100% coverage over the same week. The calibrated band is narrower and the forecast stays on the realised curve. Red would mark an hour outside the interval.*

What this result establishes

On the 2026 test set, the multi-resolution correction improves point accuracy and narrows the intervals by 27.8%, with empirical coverage of 96.30%, above the 95% target and above RTE's 95.09%. The gain comes from correcting the public residual, not from a forecast rebuilt from scratch.

10 Conclusion

On the 4 343 hours of the 2026 test set, RTE + LightGBM clearly outperforms the original RTE forecasts, both in accuracy and in reliability. First, it reduces mean forecast error (MAPE) by **40.75%**, from 2.99% to 1.77%. The result is even more relevant for uncertainty management: at the **same 95% coverage target**, the model produces prediction intervals that are **25.80% narrower**, while achieving better empirical coverage (95.12% versus 93.51%). In other words, we forecast more accurately, with less displayed uncertainty, without sacrificing interval reliability. The same improvement also reduces interval failures by **24.82%** and, above all, errors above the 5% threshold by **86.46%**. These results show the interest of the approach: rather than rebuilding an electricity forecasting model from scratch, we exploit the quality of RTE forecasts and learn to correct their errors. Performance on data never used for training confirms that this strategy works over the period studied.

Two limitations frame this result. The first is informational: without the RTE forecast available at 11:00, none of the models in this project reaches operational level. That availability assumption must be checked against the real publication schedule. The second is statistical: corrected residuals remain autocorrelated at lags 24, 48 and 168 h. An ARMA post-filter, or a multi-scale sequence model such as MTST, has theoretical reasons to capture that remaining structure. This does not invalidate the 2026 gain; it indicates the next step.

References

[1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel and G. M. Ljung, *Time Series Analysis: Forecasting and Control*. Wiley, 5th ed., 2015.

[2] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*. OTexts, 3rd ed., 2021.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree ", *Advances in Neural Information Processing Systems*, 2017.

[4] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani and L. Wasserman, "Distribution-Free Predictive Inference for Regression ", *Journal of the American Statistical Association*, vol. 113, no. 523, 2018.

[5] Y. Zhang, L. Ma, S. Pal, Y. Zhang and M. Coates, "Multi-resolution Time-Series Transformer for Long-term Forecasting ", *Proceedings of AISTATS*, PMLR vol. 238, 2024.

[6] Y. Nie, N. H. Nguyen, P. Sinthong and J. Kalagnanam, "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers ", *International Conference on Learning Representations*, 2023.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention Is All You Need ", *Advances in Neural Information Processing Systems*, 2017.

[8] A. Zeng, M. Chen, L. Zhang and Q. Xu, "Are Transformers Effective for Time Series Forecasting? ", *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.